



GRADO EN INGENIERÍA INFORMÁTICA

TRABAJO DE FINAL DE GRADO

Diseño e implementación de una plataforma de monitorización y alerta para entornos de Industria 4.0

Autor/a:

Joan-Llop MARCOS ROGER

Supervisor/a:

Mihai Catalin BUDAI

Tutor/a académico/a:

Miguel MATEY SANZ

Fecha de lectura: – de — de 20—
Curso académico 2025/2026

Resumen

La digitalización de los entornos industriales requiere soluciones capaces de supervisar dispositivos de forma remota y detectar incidencias de manera temprana. En este Trabajo de Final de Grado se presenta el diseño e implementación de una plataforma para la monitorización y gestión de máquinas industriales basada en una arquitectura cliente-servidor.

La solución desarrollada permite recopilar métricas procedentes de dispositivos desplegados en planta, almacenarlas de forma centralizada y generar alertas automáticas a partir de reglas configurables definidas por el usuario. La plataforma incorpora una interfaz web para la gestión de *gateways*, conectores, máquinas y alertas, facilitando la supervisión de la infraestructura monitorizada.

Para su desarrollo se han utilizado tecnologías como NestJS, TypeScript, PostgreSQL, TypeORM y Docker. Finalmente, se realizaron pruebas unitarias, de integración y de validación funcional que permitieron verificar el correcto funcionamiento del sistema. Los resultados obtenidos demuestran la viabilidad de la solución como base para futuros sistemas de monitorización industrial y mantenimiento predictivo.

Palabras clave

Monitorización industrial, Industria 4.0, Internet de las Cosas (IoT), Alertas automáticas, Sistemas Edge

Keywords

Industrial Monitoring, Industry 4.0, Internet of Things (IoT), Automated Alerts, Edge Computing

Índice general

1. Introducción	7
1.1. Contexto y motivación del proyecto	7
1.2. Objetivo y alcance del producto	8
1.3. Objetivo y alcance del proyecto	9
1.4. Descripción detallada del desarrollo del proyecto	10
1.5. Estructura de la memoria	11
2. Planificación del proyecto	13
2.1. Metodología y planificación temporal del proyecto	13
2.2. Seguimiento del proyecto	16
2.2.1. Metodología de seguimiento	16
2.2.2. Seguimiento por <i>sprints</i>	17
2.2.3. Desviaciones respecto al backlog inicial	20
2.2.4. Resolución de incidencias	21
2.2.5. Evaluación final del seguimiento	21
2.3. Costes	21
2.3.1. Costes de recursos humanos	21
2.3.2. Costes de recursos tecnológicos	23
2.3.3. Coste total del proyecto	24

2.4. Riesgos	25
3. Análisis	27
3.1. Investigación de requisitos	27
3.2. Definición de requisitos	28
3.2.1. Requisitos funcionales	28
3.2.2. Requisitos de datos	28
3.2.3. Requisitos no funcionales	34
3.3. Análisis de requisitos	34
3.3.1. Modelo conceptual	37
3.3.2. Análisis de los procesos principales	37
3.3.3. Análisis de alternativas	40
3.3.4. Resultados del análisis	40
4. Diseño	43
4.1. Diseño de la base de datos	43
4.1.1. Entidad <i>Gateway</i>	43
4.1.2. Entidad Connector	43
4.1.3. Entidad Machine	45
4.1.4. Entidad AttributeValue	45
4.1.5. Entidad AlertRule	45
4.1.6. Entidad Alert	46
4.1.7. Entidad User	46
4.2. Diseño de software	46
4.2.1. Arquitectura general	46
4.2.2. Patrón arquitectónico	47

4.2.3.	Inyección de dependencias	47
4.2.4.	Diseño modular	49
4.2.5.	Flujo principal de monitorización	49
4.3.	Diseño de las interfaces	49
4.3.1.	Guia de estilo visual	51
4.3.2.	Pantallas principales	51
4.4.	Diseño de la arquitectura	52
4.4.1.	Arquitectura general	52
4.4.2.	Infraestructura de persistencia	54
4.4.3.	Justificación de la arquitectura	54
5.	Implementación y pruebas	55
5.1.	Estructura del código	55
5.1.1.	Organización general	55
5.1.2.	Controladores	57
5.1.3.	Servicios	59
5.1.4.	Entidades	60
5.1.5.	DTO: Data Transfer Objects	60
5.1.6.	Interacción entre componentes	61
5.2.	Descripción técnica de la implementación	62
5.2.1.	Tecnologías y ejecución	62
5.2.2.	Persistencia y despliegue de los servicios de datos	62
5.2.3.	Métricas recogidas y consultas	63
5.2.4.	API REST	64
5.2.5.	Evaluación de umbrales y notificación	64

5.2.6. Resultados de la implementación	65
5.3. Pruebas y validación	65
5.3.1. Estrategia de pruebas	65
5.3.2. Pruebas unitarias	66
5.3.3. Pruebas de integración	66
5.3.4. Pruebas de sistema y validación funcional	67
5.3.5. Casos de prueba y resultados	67
6. Conclusiones	69
6.1. Objetivos logrados	69
6.2. Trabajo futuro	70
6.3. Conclusiones personales	70

Capítulo 1

Introducción

1.1. Contexto y motivación del proyecto

El proyecto se desarrolla en el contexto de la empresa llamada Ingeniería Plana Alta S.A [6, 7], dedicada a la automatización y digitalización para implementar en la industria 4.0. La empresa se centra en la construcción y instalación de paletizadores automáticos para el final de línea de producción. Estas líneas automáticas están formadas por una cinta transportadora y *sorters*, que transportan el producto y lo clasifican, y una serie de paletizadores autónomos al final de las cintas, que mediante brazos robóticos y soportes móviles colocan el producto en palets formando un mosaico ordenado. Dentro de la empresa también existe un departamento de informática, que mantiene y desarrolla la plataforma digital de control del sistema de final de línea y paletizadores, ConnectiveX. Esta plataforma digital se compone de diversos módulos con funciones como gestión de mantenimientos y visualización de *dashboards* de datos. Los módulos se han ido desarrollando e incluyendo a medida que han surgido nuevas necesidades, componiendo así una herramienta centralizada de gestión de maquinaria e información de los paletizadores y final de línea.

El proyecto surge al descubrir la necesidad de incluir un nuevo módulo para la correcta gestión y prevención de fallos en las máquinas que instala la empresa, tanto a nivel de hardware como de software, ya que hasta ahora, los clientes no tenían forma de determinar el estado y necesidad de mantenimiento de una máquina sin hacer comprobaciones manuales exhaustivas. Los clientes necesitaban nuevas funciones que permitieran descubrir posibles desajustes en las máquinas antes de que ocurrieran fallos críticos, ayudando a prevenir la sustitución de sistemas complejos tras malfuncionamientos en los paletizadores o cintas transportadoras. También se propone un módulo de actualizaciones automáticas *Over-The-Air*, que ayuda a prevenir fallos de software y de seguridad al instalar e actualizar automáticamente el software a la última versión. Este módulo permitirá cubrir la necesidad de ir presencialmente a las plantas de los clientes para realizar la instalación manual del software.

Este proyecto se propone porque hasta ahora la supervisión de los nodos edge desplegados en planta y la gestión de sus actualizaciones de software se realizaban mediante procedimientos mayoritariamente manuales. La información sobre el estado de los dispositivos se obtenía de

forma dispersa y las tareas de mantenimiento requerían, en determinadas ocasiones, la intervención directa de personal técnico e incluso desplazamientos físicos hasta las instalaciones. Esta situación dificultaba la detección temprana de incidencias y aumentaba el tiempo necesario para aplicar actualizaciones o resolver problemas de funcionamiento.

El proyecto consiste en desarrollar un sistema que permita monitorear el estado de salud y de conexión estos equipos y realizar actualizaciones automáticas de forma remota, aumentando la eficiencia operativa al reducir tiempos de inactividad y tiempos de resolución de problemas.

1.2. Objetivo y alcance del producto

El producto que se desarrollará en este proyecto es un sistema de supervisión y gestión remota de nodos *edge* desplegados en planta, cuyo propósito es mejorar la eficiencia y fiabilidad de las operaciones industriales al permitir el monitoreo continuo del estado de los dispositivos y sus comunicaciones y la actualización remota del software que ejecutan. El sistema permitirá a la empresa detectar de forma temprana posibles incidencias en los equipos, como sobrecargas de CPU, fallos de conectividad o consumo excesivo de memoria, reduciendo así los tiempos de inactividad y facilitando la sincronización de versiones y configuración entre todas las máquinas.

El sistema proporcionará una plataforma centralizada de gestión desde la cual los administradores podrán consultar métricas de salud de los nodos, programar actualizaciones de software automáticas y supervisar la conectividad general de la red de nodos de la planta. De esta forma, el tiempo dedicado a tareas de mantenimiento manual o desplazamientos físicos para comprobar el estado de los dispositivos disminuirá considerablemente, y se incrementará la capacidad del personal técnico para actuar de manera proactiva y preventiva frente a posibles fallos. Además, el producto permitirá mantener la infraestructura distribuida de la empresa siempre actualizada y alineada con los requisitos de seguridad y rendimiento necesarios para las operaciones críticas.

El alcance funcional del sistema incluye distintas características:

- Supervisión del estado de los nodos edge mediante la recopilación y visualización de métricas como uso de CPU, memoria y conectividad.
- Gestión de actualizaciones de software *Over-The-Air* (OTA), permitiendo enviar nuevas versiones o parches a los dispositivos de manera remota y automatizada.
- Almacenamiento y análisis de datos IoT en una base de datos diseñada para mantener el historial de métricas y facilitar consultas de rendimiento.
- Generación de alertas ante condiciones anómalas o pérdida de conexión de un nodo.

No se incluye en este proyecto la implementación de módulos de inteligencia artificial para el análisis predictivo de fallos, aunque esta funcionalidad podría plantearse en fases posteriores como evolución del sistema.

Respecto a la perspectiva organizacional, el sistema será desarrollado en el departamento de informática de la empresa, y su uso impactará principalmente en esa área, ya que se beneficiarán

de mayor disponibilidad y control de los nodos, así como mayor facilidad para la gestión del software necesario para cada uno. El personal de informática utilizará el sistema tanto para la supervisión y actualización de los nodos como para la consulta del estado general de los equipos para planificar tanto la actualización del software como la prevención de posibles incidencias o periodos de inactividad por sobrecarga. Además, el sistema se podrá integrar con los procesos internos de soporte técnico y con las políticas de actualización de software establecidas por la empresa.

Desde el punto de vista tecnológico, el sistema se conectará con los nodos edge desplegados en planta, que se comunican con las máquinas mediante el protocolo OPC UA (Open Platform Communications Unified Architecture [5, 13]) y con el nodo central mediante el protocolo MQTT (Message Queuing Telemetry Transport [11]). Estos nodos enviarán sus métricas a la aplicación por el nodo central, donde se almacenarán en una base de datos dependiendo del tipo y frecuencia de los datos. El sistema también estará vinculado a un *backend* desarrollado con APIs REST (Application Programming Interface, Representational State Transfer [3]), encargado de gestionar la comunicación entre los dispositivos, la base de datos y las herramientas de análisis y alertas.

1.3. Objetivo y alcance del proyecto

El principal objetivo del proyecto es desarrollar un sistema que permita supervisar el estado de los nodos remotos instalados en la planta industrial y gestionar sus actualizaciones de software de forma remota, garantizando así un mantenimiento más eficiente y una mayor disponibilidad operativa de los equipos.

Este objetivo se puede desglosar en los siguientes subobjetivos:

- Diseñar y desarrollar servicios que recopilen métricas de salud de los nodos.
- Implementar un sistema de actualizaciones automáticas *Over-The-Air* que facilite la instalación remota de nuevas versiones de software.
- Crear e integrar APIs REST que permitan la comunicación fluida entre los dispositivos industriales y la plataforma central.
- Desarrollar la lógica de negocio y la estructura de bases de datos necesarias para el almacenamiento, consulta y análisis de la información recopilada.
- Realizar pruebas unitarias y documentar adecuadamente el funcionamiento del software desarrollado

El alcance del proyecto abarca la participación directa en el desarrollo e integración de estas funcionalidades dentro del sistema existente de la empresa, así como la colaboración con otros miembros del equipo en tareas de mantenimiento, pruebas e implementación de mejoras orientadas a la fiabilidad y escalabilidad del sistema.

1.4. Descripción detallada del desarrollo del proyecto

La empresa se encuentra desarrollando una plataforma para controlar el IoT en la maquinaria que instalan y digitalizan y se han encontrado con la necesidad de implementar un módulo que permita la transmisión de información entre las máquinas y la plataforma de control, de forma que se pueda tanto consultar información sobre métricas como enviar nuevas versiones de software para realizar actualizaciones automáticamente. El programa también debe trabajar con los datos que reciba. Las métricas que superen los umbrales que defina el administrador o usuario de la plataforma deben ser detectadas y deben lanzar la alerta correspondiente.

Para implementar la solución a esta necesidad, el departamento de informática de la empresa evaluó las distintas tecnologías que se podían usar para la comunicación y la implementación de todo el software y eligió las siguientes:

- MQTT: Es un protocolo de comunicación para IoT, ligero, que usa el formato *publish/subscribe*. Suele usarse en domótica y aplicaciones IoT, en especial en comunicaciones máquina a máquina. En este proyecto se empleará para la comunicación entre el nodo central y los nodos *edge*.
- OPC UA: Es un protocolo de comunicación estándar para sistemas en entornos industriales que mejora la seguridad, la fiabilidad y la interoperabilidad en los intercambios de datos. En este proyecto se empleará para la comunicación entre los nodos *edge* y las máquinas de la planta.
- NestJs [9]: Se trata de un *framework* de servidor de Node.Js que sirve para crear aplicaciones *backend* eficientes, fácilmente escalables y fáciles de mantener. En el proyecto se empleará para desarrollar toda la parte de *backend*.
- Angular [17]: Es un *framework* basado en componentes que sirve para crear el *frontend* de aplicaciones web escalables. En el proyecto se empleará para desarrollar la base del *frontend* de la aplicación.
- Ionic [14]: Es un *framework* de *frontend* que sirve para desarrollar aplicaciones híbridas, tanto para web como para iOS y Android. En el proyecto se usará para construir el *frontend* con componentes adaptativos para todos los dispositivos.
- PostgreSQL [4]: Se trata de una base de datos SQL relacional comunmente usada por su alta fiabilidad y flexibilidad para todo tipo de aplicaciones y usos. En este proyecto se empleará para almacenar datos con los que trabaja la aplicación, como las máquinas o las alertas.
- Kubernetes: Se trata de una plataforma de automatización y administración de micro-servicios y contenedores, que permite la orquestación de infraestructuras de cómputo, redes y almacenamiento. En el proyecto se usará para los despliegues y actualizaciones automáticos de las máquinas.
- Docker [15]: Es una tecnología de virtualización en contenedores livianos y modulares, muy fácilmente adaptables a distintos entornos, cosa que lo hace ideal para aplicaciones para la nube. En este proyecto se usará para ejecutar contenedores con las bases de datos y la API REST del *backend*.

1.5. Estructura de la memoria

La presente memoria se encuentra organizada en varios capítulos que describen de forma progresiva las distintas fases del desarrollo del proyecto, desde el análisis inicial del problema hasta las conclusiones finales obtenidas.

El **Capítulo 2** aborda el estudio previo realizado antes del desarrollo de la solución. En este capítulo se describen los antecedentes del problema, las tecnologías analizadas y el proceso de investigación y obtención de requisitos.

En el **Capítulo 3** se desarrolla el análisis del sistema. Se presentan los requisitos funcionales y no funcionales identificados, los casos de uso principales y los modelos conceptuales utilizados para definir el comportamiento esperado de la aplicación.

El **Capítulo 4** recoge el diseño de la solución propuesta. En él se describen el diseño de la base de datos, la arquitectura software, la organización de los distintos componentes del sistema y la arquitectura de despliegue utilizada.

El **Capítulo 5** se centra en la implementación del proyecto. Se describe la estructura del código desarrollado, las decisiones técnicas adoptadas durante la implementación y el proceso de verificación y validación llevado a cabo para comprobar el correcto funcionamiento de la solución.

Finalmente, el **Capítulo 6** presenta las conclusiones del trabajo, evaluando el grado de cumplimiento de los objetivos planteados, las competencias adquiridas durante el desarrollo, las lecciones aprendidas y las posibles líneas de trabajo futuro.

Capítulo 2

Planificación del proyecto

2.1. Metodología y planificación temporal del proyecto

Para el desarrollo del proyecto se ha adoptado una metodología ágil basada en *Scrum* [1], dado que se trata de un entorno en el que los requisitos y prioridades pueden evolucionar conforme se avanza en la implementación del programa y en la integración con los sistemas industriales de la empresa. En esta instancia se ha asignado como Product Owner y como *Scrum Master* a Mihai Catalin Budai, el CIO de la empresa IPLA, y por otra parte, se ha asignado como equipo de desarrollo a Joan-Llop Marcos Roger, el estudiante en prácticas. La metodología se ha aplicado llevando a cabo *sprints* de 3 semanas en los que se crea y expande una parte fundamental del proyecto en cada uno. Para el seguimiento del desarrollo, se han realizado reuniones diarias en las que cada miembro del equipo expresa el avance en su tarea respecto a la reunión anterior y su trabajo pendiente. Finalmente, tras completar las tareas de cada sprint, se revisan completamente antes de pasar al siguiente.

El proyecto contempla 300 horas presenciales en la empresa, distribuidas a lo largo de 15 semanas, con una dedicación de 20 horas semanales. El desarrollo se organizará en 5 *sprints* de 3 semanas de duración, lo que se traduce a 60 horas por *sprint*. Cada *sprint* tendrá un objetivo claro y entregables concretos, de acuerdo con la pila del producto.

Para la correcta estimación del proyecto se ha dividido el trabajo en tareas, y se han designado las horas basándose en la prioridad y complejidad de cada tarea (Ver Cuadro 2.1). Para determinar la cantidad de horas que serían necesarias para completar una tarea, se ha investigado el coste temporal de tareas similares en proyectos realizados previamente en la empresa y se han adaptado a la planificación actual. Por otra parte, la prioridad de las tareas se ha pensado centrándose en la importancia de lo que pretenden desarrollar sobre el producto. En caso de tratarse de un pilar fundamental para el producto se le ha asignado una alta prioridad, y en caso de tratarse de una característica no bloqueante o no decisiva se ha rebajado su prioridad.

- **P1: Configuración y arquitectura base:** Esta tarea del backlog representa la necesidad de definir un entorno de desarrollo y configurarlo, además de concretar una estructura para el *backend*, crear un repositorio para el seguimiento de versiones y asegurar el fun-

ID	Tarea	Prioridad	Horas
P1.1	Definir la arquitectura general del sistema	Alta	12
P1.2	Configurar el entorno de desarrollo y preparar el repositorio	Alta	12
P2.1	Implementar la API del <i>backend</i>	Alta	34
P2.2	Conectar la API con la BBDD	Alta	10
P3.1	Configurar e integrar la base de datos con Docker Compose	Alta	12
P4.1	Desarrollar el <i>frontend</i> de la API base	Media	14
P4.2	Desarrollar el <i>frontend</i> de los módulos de alertas	Media	10
P4.3	Desarrollar el <i>frontend</i> del sistema OTA	Media	8
P5.1	Implementar el sistema de actualizaciones OTA	Alta	20
P6.1	Desarrollar el módulo de gestión de alertas	Alta	24
P6.2	Desarrollar el módulo de ingesta de alertas	Alta	22
P7.1	Realizar pruebas unitarias para la API	Alta	10
P7.2	Realizar pruebas unitarias para el módulo de gestión de alertas	Alta	6
P7.3	Realizar pruebas unitarias para el módulo de ingesta de alertas	Alta	8
P7.4	Realizar pruebas manuales para el sistema OTA	Alta	8
P7.5	Realizar pruebas manuales para el <i>frontend</i>	Alta	8
P7.6	Realizar pruebas de integración para la aplicación completa	Alta	14
P7.7	Corregir los errores encontrados en las pruebas	Alta	22
P8.1	Documentar la configuración del entorno de desarrollo	Media	8
P8.2	Documentar el desarrollo y uso de la API	Media	6
P8.2	Documentar el desarrollo y uso del módulo de alertas	Media	8
P8.3	Documentar el desarrollo y uso del <i>frontend</i> de la API base	Media	8
P8.4	Documentar el desarrollo y uso del <i>frontend</i> de los módulos de alertas	Media	8
P8.5	Documentar el desarrollo y uso del <i>frontend</i> del sistema OTA	Media	8
Total			300

Cuadro 2.1: Primera versión de la pila del producto

cionamiento de todas las herramientas antes de empezar el desarrollo del proyecto.

- **P2: API REST:** Esta tarea representa la necesidad de crear una base de funciones y servicios que la aplicación tiene que tener en forma de API REST. Los diversos servicios que incluye son la gestión de *gateways*, conectores y máquinas.
- **P3: Almacenamiento:** Esta tarea representa la necesidad de diseñar, implementar y utilizar una base de datos para cubrir las necesidades de la aplicación, incluyendo el apartado de alertas, en el cual debe ser capaz de almacenar las métricas y las alertas que se hayan lanzado.
- **P4: Visualización:** Este elemento representa la necesidad de desarrollar un panel o interfaz para permitir la visualización, consulta y uso de las distintas características y datos de la aplicación.
- **P5: Sistema OTA:** Esta tarea representa la necesidad de crear un módulo que permita actualizar de forma remota el software de los nodos *edge*. Este sistema debe permitir controlar las actualizaciones remotas por cada nodo *edge* individual.
- **P6: Alertas:** Esta tarea representa la necesidad de implementar un módulo que permita configurar, gestionar y consultar alertas para las distintas métricas que emiten las máquinas registradas en la aplicación. Debe generar las alertas, consultar las alertas pasadas y filtrar por severidad en la búsqueda de alertas.
- **P7: Pruebas e integración:** Esta tarea representa la necesidad de crear pruebas que validen el correcto funcionamiento de la aplicación, tanto a nivel unitario como a nivel de integración entre los distintos componentes de la aplicación.
- **P8: Documentación:** Esta tarea representa la necesidad de redactar documentos con información sobre el uso de la aplicación y los procesos seguidos para implementarla.

La pila del producto definida incluye las tareas correspondientes a los requisitos funcionales principales identificados durante la fase de análisis, así como otras tareas necesarias para completar el proyecto, como la realización de pruebas o la elaboración de la documentación técnica.

Por otra parte, el primer *sprint* tuvo como objetivo establecer la infraestructura básica necesaria para el desarrollo posterior del sistema. Esto se llevó a cabo definiendo la arquitectura general, preparando el entorno de desarrollo y implementando una primera versión funcional de la API que permita recibir métricas simuladas desde los nodos. Las tareas incluidas en esta primera iteración se detallan en el Cuadro 2.2.

- T1: Estudiar el problema, buscar los requisitos que debe cumplir el producto para solucionarlo y definir el camino a seguir para desarrollar la solución.
- T2: Definir cada parte del proyecto y cómo interaccionará con las demás. Se define tanto los componentes del *backend* como las interacciones de nodos a *backend* y las características de las bases de datos. Corresponde al requisito P1 del producto.

Tarea	Descripción	Horas estimadas
T1	Estudiar el problema y definir requisitos del producto.	12
T2	Definir la arquitectura general del sistema (<i>backend</i> , nodos y base de datos).	14
T3	Configurar el entorno de desarrollo (Docker, Git, VS Code) y preparar la estructura inicial del repositorio.	12
T4	Configurar e integrar la base de datos PostgreSQL con Docker Compose.	14
T5	Documentar el diseño y la configuración del entorno de desarrollo.	8
Total		60

Cuadro 2.2: Primera pila del *sprint* (*sprint* Backlog 1)

- T3: Configurar cada aplicación para el funcionamiento para el proyecto, como por ejemplo, preparar los plugins de VS Code y crear un nuevo repositorio de Git y hacer un commit inicial para definir la estructura que tendrá el proyecto. Corresponde al requisito P1 del producto.
- T4: Configurar e implementar en el docker configurado en la T1 la base de datos que se definió en la T2. Crear la base de datos y contenerizarla correctamente para poder usarla en la API de la aplicación. Corresponde al requisito P3 del producto.
- T5: Documentar la configuración del entorno de desarrollo y toda la arquitectura e implementación del *backend*, nodos y bases de datos. Corresponde al requisito P8 del producto.

La finalización satisfactoria de las tareas del primer *sprint* permitió disponer de una base sólida sobre la que construir las funcionalidades desarrolladas durante los *sprints* posteriores.

2.2. Seguimiento del proyecto

2.2.1. Metodología de seguimiento

El seguimiento del proyecto se realizó basado en su planificación ágil dividida en *sprints*, permitiendo controlar periódicamente el avance de las tareas y adaptar la planificación a las necesidades que fueron surgiendo durante el desarrollo.

Al inicio de cada *sprint* se definieron los objetivos a alcanzar y las funcionalidades que debían implementarse. Al finalizar cada iteración se realizó una revisión del trabajo desarrollado, evaluando el grado de cumplimiento de los objetivos establecidos e identificando posibles desviaciones respecto a la planificación prevista.

Tarea	Descripción	Horas estimadas
T1	Implementar la API base del <i>backend</i> .	34
T2	Conectar la API con la BBDD	10
T3	Realizar pruebas unitarias para la API	10
T4	Documentar los pasos realizados	6
Total		60

Cuadro 2.3: Pila del *sprint* 2

Este enfoque permitió disponer de una visión continua del estado del proyecto y facilitar la toma de decisiones durante el desarrollo.

2.2.2. Seguimiento por *sprints*

***sprint* 1 (13/10/25 - 03/11/25): Investigación, definición de requisitos y configuración inicial**

Durante este *sprint* se realizó el estudio del problema, la investigación de tecnologías y la recopilación de requisitos funcionales y no funcionales. Después se configuró el entorno de desarrollo y se preparó el repositorio del proyecto en git. También se creó la base de datos contenerizada del proyecto. Finalmente, se redactó documentación sobre los pasos seguidos.

La pila correspondiente a este *sprint* es el Cuadro 2.2, que se muestra anteriormente.

Resultado de la revisión:

El product owner y el equipo de desarrollo validaron el alcance y requisitos del producto definidos y se dió luz verde a empezar a desarrollar el proyecto con las características planteadas.

sprint* 2 (03/11/25 - 24/11/25): Implementación de la infraestructura básica del *backend

Durante este *sprint* se desarrolló la estructura principal del *backend* y se configuró la persistencia de datos. Se empezó por desarrollar las funcionalidades necesarias para la API base de la aplicación y después se conectaron dichas funcionalidades con la BBDD, para que tuvieran efecto persistente. Un ejemplo es el registro de una nueva máquina en el sistema, que requiere una petición a la API que cree una nueva entidad de tipo máquina con los datos introducidos y la guarde en la base de datos. Tras ello, se realizaron pruebas unitarias para la API para confirmar el funcionamiento de cada función y, finalmente, se documentó el desarrollo y uso de las nuevas características de la aplicación.

Resultado de la revisión:

En la reunión de revisión y progreso del proyecto se investigaron los resultados de las pruebas

Tarea	Descripción	Horas estimadas
T1	Desarrollar el módulo de gestión de alertas.	24
T2	Desarrollar el módulo de ingesta de alertas	22
T3	Realizar pruebas unitarias para los nuevos módulos	6
T4	Documentar los pasos realizados	8
Total		60

Cuadro 2.4: Pila del *sprint* 3

unitarias y si la API cumplía con los requisitos del producto y se determinó que se había desarrollado una versión funcional capaz de almacenar información en la base de datos.

***sprint* 3 (24/11/25 - 16/12/25): Desarrollo de funcionalidades de alertas**

En esta fase se empezó a desarrollar la parte de las alertas que corresponde al requisito P6. En este sprint se planeaba desarrollar los módulos que añadirían la funcionalidad de las alertas a la aplicación: los módulos de gestión de alertas y los de ingesta de alertas. El primero de ellos permite buscar alertas y hacer la comprobación de si superan los umbrales definidos por máquina y el segundo es el encargado de recibir las alertas, guardarlas en la base de datos y llamar al servicio de comprobación del otro módulo. También se desarrollaron las pruebas unitarias de los nuevos módulos y se documentaron los procedimientos y uso de cada uno.

Desviaciones en la planificación:

Durante el sprint la tarea T1: “Desarrollar el módulo de gestión de alertas” tomó un tiempo mayor al estimado, de forma que, al terminarla, no quedaban horas suficientes para desarrollar completamente la tarea T2: “Desarrollar el módulo de ingesta de alertas”, por lo que se empezó a desarrollar pero no se pudo completar en este sprint. De igual forma sucedió con las tareas T3 y T4 que, al tener una dependencia en la tarea T2, se tuvieron que posponer al siguiente sprint, ya que no se podía documentar ni se podía probar código no desarrollado aún, aunque la parte del módulo de gestión de alertas sí que se completó, así como la documentación y pruebas unitarias del mismo.

Resultado de la revisión:

En la revisión el *Scrum Master* tomó la decisión de posponer las tareas que quedaron pendientes para el siguiente sprint, ya que se trataba de un requisito muy importante para el producto y no podía dejarse para el último sprint debido al trabajo pendiente que aún quedaba en el proyecto. Tras ello se evaluó el resultado del desarrollo del sprint y se determinó que la plataforma permitía gestionar correctamente las alertas de las máquinas y evaluarlas frente a los umbrales establecidos, a falta de recibir alertas y almacenarlas, que es tarea del módulo de ingesta, que quedaba para desarrollar en el sprint siguiente.

Tarea	Descripción	Horas estimadas
T1	Desarrollar el módulo de ingesta de alertas.	10
T2	Desarrollar el <i>frontend</i> de la API base	16
T3	Realizar pruebas unitarias para el módulo de ingesta de alertas	8
T4	Realizar pruebas manuales para el <i>frontend</i>	16
T5	Documentar los pasos realizados	10
Total		60

Cuadro 2.5: Pila del *sprint* 4

***sprint* 4 (16/12/25 - 12/01/26): Desarrollo del *frontend* y finalización del *backend* de ingesta de alertas**

Durante esta iteración se terminó de desarrollar el módulo de ingesta de alertas y se realizaron pruebas unitarias para comprobar su funcionamiento. Tras ello, se desarrolló el *frontend* de toda la parte de la API base, incluyendo interfaces como las de inicio de sesión o de registro de máquinas. Posteriormente se realizaron pruebas manuales para comprobar que las acciones del *frontend* se comunicaban con el *backend* y se ejecutaban correctamente, y después se documentaron el desarrollo y el uso de el módulo de ingesta de alertas y del *frontend* de la API.

Resultado de la revisión:

Durante la reunión de revisión se evaluó el resultado del desarrollo del *sprint* y se probaron las funciones implementadas, así como el *frontend* que se había desarrollado. Una vez probado, se determinó que el resultado había sido satisfactorio y que cumplía con los requisitos del product backlog (Requisito P6 y parte del requisito P4).

Después, el *Scrum Master* y el equipo de desarrollo analizaron el progreso del proyecto y los elementos del backlog que aún no se habían desarrollado y tomaron la decisión de **eliminar el requisito P5: Sistema OTA del backlog**, ya que dada la complejidad y la estimación horaria del mismo, no era factible completarlo en el tiempo restante. No obstante, dado que había sido un elemento interesante para la aplicación, se quedó propuesto como trabajo futuro.

Requisitos eliminados del backlog:

- P5. Sistema OTA: Requisito que consistía en implementar un módulo para la actualización *Over-The-Air* (OTA) del software de los nodos edge. Esto implicaba desarrollo del módulo de *backend* y configuración del despliegue con Kubernetes. Por falta de tiempo debido a estimaciones incorrectas en la planificación, este requisito queda fuera del alcance del proyecto y se propone como trabajo futuro.

Tarea	Descripción	Horas estimadas
T1	Desarrollar el <i>frontend</i> del sistema de alertas.	10
T2	Realizar pruebas de integración para la aplicación completa	16
T3	Corregir los errores encontrados en las pruebas	24
T4	Documentar los pasos realizados	10
Total		60

Cuadro 2.6: Pila del *sprint* 5

***sprint* 5 (12/01/26 - 02/02/26): Interfaz web y pruebas de la aplicación**

Durante este *sprint* se desarrolló el *frontend* del sistema de alertas, con lo que la interfaz web quedó completada, y se realizaron pruebas manuales de integración entre el *frontend* y el *backend*. Al hacer las pruebas se detectaron y depuraron los errores que habían quedado en el código desarrollado previamente, con lo que quedó la aplicación con la funcionalidad completa. Después, se documentó el funcionamiento y uso del *frontend* del sistema de alertas de la aplicación.

Resultado de la revisión:

En la revisión se evaluaron las implementaciones y resoluciones de cada tarea y se determinó que se consiguió implementar el *frontend* del sistema y perfeccionar el código desarrollado anteriormente. Tras ello, se revisó el estado general de la aplicación, que, a pesar de sufrir desviaciones respecto a la estimación temporal inicial y a causa de ello sufrir cambios en el backlog, cumplía satisfactoriamente con los requisitos definidos del producto. Finalmente, se propusieron mejoras que quedarían incluidas como trabajo futuro.

2.2.3. Desviaciones respecto al backlog inicial

Durante el desarrollo surgieron algunas desviaciones respecto a la estimación temporal original.

La principal desviación estuvo relacionada con el tiempo necesario para implementar y validar determinadas funcionalidades del *backend* asociadas al almacenamiento de métricas y a la generación automática de alertas, correspondientes al módulo de gestión de alertas. Estas tareas requirieron un esfuerzo superior al inicialmente previsto debido a la necesidad de garantizar la correcta integración entre los distintos módulos del sistema.

Como consecuencia, la implementación de un sistema de actualizaciones remotas *Over-The-Air* (OTA) fue pospuesta para futuras versiones dado que su incorporación quedó fuera del alcance temporal disponible para el proyecto.

2.2.4. Resolución de incidencias

Los problemas detectados durante el desarrollo fueron gestionados mediante revisiones y reuniones periódicas y ajustes de planificación entre *sprints*.

La estructura modular de la aplicación facilitó la resolución de incidencias, ya que permitió aislar los distintos componentes y corregir errores sin afectar al resto del sistema.

Asimismo, las pruebas unitarias realizadas de forma continua contribuyeron a detectar problemas en el desarrollo, reduciendo el impacto sobre las etapas posteriores del proyecto. También, las pruebas de integración facilitaron la detección de problemas entre los sistemas de *frontend* y *backend* de la aplicación, cosa que ayudó a perfeccionar el funcionamiento de la totalidad del proyecto.

2.2.5. Evaluación final del seguimiento

El seguimiento realizado durante el proyecto permitió mantener un control continuo sobre el avance de las tareas y detectar tempranamente posibles desviaciones respecto a la planificación establecida.

Aunque algunas funcionalidades previstas inicialmente tuvieron que ser pospuestas debido a las limitaciones temporales del proyecto, los objetivos principales definidos durante la fase de análisis fueron alcanzados satisfactoriamente y la solución desarrollada cumple los requisitos funcionales establecidos.

El Cuadro 2.7 representa el backlog final del producto, actualizado teniendo en cuenta las desviaciones y el coste temporal exacto de cada tarea.

2.3. Costes

En este capítulo se realiza una estimación de los costes asociados al desarrollo del proyecto. El objetivo de este análisis no es reflejar el coste real asumido durante la realización del Trabajo de Final de Grado, sino estimar el coste que tendría llevar a cabo un proyecto equivalente en un entorno profesional.

Para ello se han considerado los costes de recursos humanos y los de los recursos tecnológicos empleados durante el desarrollo.

2.3.1. Costes de recursos humanos

El desarrollo del proyecto requirió una dedicación total de 300 horas de trabajo distribuidas a lo largo de las prácticas realizadas en la empresa.

ID	Tarea	Prioridad	Horas
P0.1	Estudiar el problema y definir requisitos del producto	Alta	12
P1.1	Definir la arquitectura general del sistema	Alta	14
P1.2	Configurar el entorno de desarrollo y preparar el repositorio	Alta	12
P2.1	Implementar la API del <i>backend</i>	Alta	34
P2.2	Conectar la API con la BBDD	Alta	10
P3.1	Configurar e integrar la base de datos con Docker Compose	Alta	14
P4.1	Desarrollar el <i>frontend</i> de la API base	Media	16
P4.2	Desarrollar el <i>frontend</i> de los módulos de alertas	Media	10
P6.1	Desarrollar el módulo de gestión de alertas	Alta	24
P6.2	Desarrollar el módulo de ingesta de alertas	Alta	32
P7.1	Realizar pruebas unitarias para la API	Alta	10
P7.2	Realizar pruebas unitarias para el módulo de gestión de alertas	Alta	6
P7.3	Realizar pruebas unitarias para el módulo de ingesta de alertas	Alta	8
P7.4	Realizar pruebas manuales para el <i>frontend</i>	Alta	16
P7.5	Realizar pruebas de integración para la aplicación completa	Alta	16
P7.6	Corregir los errores encontrados en las pruebas	Alta	24
P8.1	Documentar la configuración del entorno de desarrollo	Media	8
P8.2	Documentar el desarrollo y uso de la API	Media	6
P8.3	Documentar el desarrollo y uso del módulo de alertas	Media	8
P8.4	Documentar el desarrollo y uso del <i>frontend</i> de la API base	Media	10
P8.5	Documentar el desarrollo y uso del <i>frontend</i> de los módulos de alertas	Media	10
Total			300

Cuadro 2.7: Pila del producto tras la finalización del proyecto

Para estimar el coste profesional del desarrollo se ha considerado un perfil de desarrollador junior. El salario medio de un desarrollador junior es de 22.000 € anuales [16, 2, 8], trabajando 40 horas a la semana, lo que se traduce en un coste por hora de 10,58 € [12].

Perfil	Horas	Coste hora (€)	Coste total (€)
Desarrollador software	300	10,58	3.174

Cuadro 2.8: Costes de desarrollo

Además del desarrollo, en el proyecto se considera un perfil senior, que dedica aproximadamente 2 horas semanales a la supervisión del desarrollador junior y realiza tareas de seguimiento, revisión y validación de resultados. El salario medio de un desarrollador senior es de 38.000 € al año [16, 2, 8], lo que se traduce a 18,27 €/hora [12].

Se estima una dedicación aproximada de 30 horas a lo largo de todo el proyecto, que junto al coste horario de 18,27 €/hora suma lo siguiente:

Perfil	Horas	Coste hora (€)	Coste total (€)
Supervisor técnico (CIO)	30	18,27	548,1

Cuadro 2.9: Costes de supervisión

A parte, se deben añadir el monto de costes directamente asociados a la contratación como la Seguridad Social, que suponen el 20 % y otros costes indirectos, como la infraestructura y costes de gestión y suministros, que suponen el 10 %.

Costes de desarrollo: $3.174 + (3.174 * 30 \%) = 4.126,2 \text{ €}$

Costes de supervisión: $548,1 + (548,1 * 30 \%) = 712,53 \text{ €}$

Por tanto, el coste real de recursos humanos asciende a:

$$4.126,2 + 712,53 = 4.838,73 \text{ €}$$

2.3.2. Costes de recursos tecnológicos

Durante el desarrollo se utilizaron diferentes herramientas software y hardware.

En cuanto al hardware, se empleó un ordenador Apple Mac Mini utilizado como estación de trabajo principal para el desarrollo del proyecto. Considerando un coste aproximado de adquisición de 700 € y una vida útil estimada de cinco años, únicamente se imputa al proyecto la parte proporcional correspondiente a las 300 horas de trabajo realizadas.

Concepto	Coste imputado (€)
Mac Mini (amortización proporcional)	35

Cuadro 2.10: Costes hardware

Respecto al software utilizado, la mayoría de herramientas empleadas disponen de licencias gratuitas para desarrollo:

- Visual Studio Code.
- Docker.
- PostgreSQL.
- pgAdmin.
- Postman.

Asimismo, las reuniones de seguimiento se realizaron mediante Microsoft Teams, incluido dentro de la suscripción Microsoft 365 Empresa Premium [10] ya contratada por la empresa. Este servicio no fue adquirido específicamente para el proyecto, pero al ser usado para el desarrollo, se imputa al proyecto la parte proporcional de la licencia anual por usuario contratada por la empresa. El coste anual asciende a 332,76 €, por lo que imputando solamente la parte proporcional calculando el coste por hora, el coste es 22,80 € teniendo en cuenta los dos usuarios en el proyecto y las 300 horas de duración.

Concepto	Coste imputado (€)
Licencia Microsoft 365 Empresa Premium	22,80

Cuadro 2.11: Costes software

Por tanto, el coste tecnológico completo imputable al proyecto asciende a:

$$35 + 22,80 = 57,80\text{€}$$

2.3.3. Coste total del proyecto

El Cuadro 2.12 resume la estimación económica global del proyecto.

Concepto	Coste (€)
Recursos humanos	4.838,73
Recursos tecnológicos	57,80
Total	4.896,53

Cuadro 2.12: Resumen de costes del proyecto

Por tanto, se estima que el coste total de desarrollo de una solución equivalente en un entorno profesional sería de aproximadamente 5.000 €, siendo los recursos humanos el principal componente económico del proyecto.

2.4. Riesgos

En cuanto a los riesgos, se ha realizado un análisis inicial de los principales riesgos identificados en las fases de planificación y diseño, evaluando su impacto potencial y la probabilidad de ocurrencia. Este análisis permite anticipar posibles incidencias y definir estrategias de mitigación adecuadas que minimicen sus efectos sobre el desarrollo. En el Cuadro 2.13 se resumen los riesgos más relevantes detectados, junto con las medidas preventivas o correctivas propuestas para cada caso.

Durante el desarrollo del proyecto, debido a la estimación temporal incorrecta de los requisitos del producto, se dió una situación en la que la limitación temporal de 300 horas no permitía desarrollar completamente todos los requisitos propuestos inicialmente, por lo que se tuvo que aplicar la estrategia de mitigación correspondiente y reducir el alcance del producto, es decir, retirar los requisitos menos importantes para el Product Owner para poder priorizar otros más críticos.

Por otra parte, se demostró que las estrategias de prevención cumplieron su función, ya que no se dió ninguno de los riesgos previstos ni no previstos. Gracias a la aplicación de las estrategias de prevención se pudieron evitar situaciones que comprometieran el desarrollo del producto.

Riesgo	Descripción	Impacto	Prob.	Estrategia de prevención	Estrategia de mitigación
Fallos en la comunicación con los nodos	Posibles errores en la transmisión de datos entre los nodos edge y el servidor central, debidos a latencias o configuraciones incorrectas.	Alto	Media	Asegurar el formato homogéneo de los datos que se transmiten y reducir interferencias de conexión que aumenten la latencia.	Buscar y corregir los errores que deterioran o impiden la comunicación.
Incompatibilidades con protocolos industriales (MQTT/OPC UA)	Diferencias de versión o parámetros de conexión que impidan el intercambio correcto de información entre sistemas.	Medio	Media	Validar las especificaciones técnicas de los protocolos y efectuar pruebas tempranas de interoperabilidad.	Investigar y asignar nuevos protocolos que sí sean compatibles.
Curva de aprendizaje de nuevas herramientas	El uso de nuevas tecnologías puede requerir un tiempo inicial de adaptación.	Medio	Alta	Dedicar un <i>sprint</i> inicial a la configuración del entorno y escribir documentación para reducir la incertidumbre.	Realizar una formación breve sobre las herramientas no conocidas por el equipo de desarrollo.
Cambios en los requisitos del cliente	Ajustes funcionales solicitados por la empresa durante el desarrollo, que puedan alterar la planificación prevista.	Medio	Media	Revisar y priorizar de forma continua el backlog con el Product Owner en cada reunión de <i>sprint</i> .	Redirigir los esfuerzos hacia el desarrollo de los nuevos requisitos solicitados por la empresa.
Limitación temporal (300 h presenciales)	El tiempo disponible puede resultar insuficiente si surgen incidencias técnicas o ampliaciones de alcance.	Alto	Media	Priorizar funcionalidades críticas y realizar entregas iterativas que aseguren un producto funcional en cada <i>sprint</i> .	Reducir el alcance funcional del producto, priorizando que cumpla correctamente los requisitos más interesantes para el Product Owner.

Cuadro 2.13: Análisis de riesgos inicial del proyecto

Capítulo 3

Análisis

3.1. Investigación de requisitos

La fase de investigación de requisitos tuvo como objetivo comprender el problema que se pretendía resolver, identificar las necesidades reales de la empresa y determinar las funcionalidades que debía proporcionar la solución desarrollada. Para ello, se estudió tanto la situación existente en la organización como la infraestructura tecnológica utilizada en el entorno industrial donde iba a integrarse el sistema.

Para la obtención de requisitos se emplearon varias técnicas complementarias. En primer lugar, se realizaron entrevistas y reuniones periódicas con el tutor de empresa y con los miembros del departamento de informática implicados en el proyecto. Estas reuniones permitieron conocer los procesos de trabajo existentes, las necesidades operativas de la organización y las expectativas sobre el sistema a desarrollar. Además, durante las reuniones de seguimiento se validaban las decisiones tomadas y se refinaban los requisitos conforme avanzaba el desarrollo.

En segundo lugar, se llevó a cabo una revisión del sistema existente y de la arquitectura tecnológica utilizada por la empresa. Se analizaron los nodos industriales desplegados en planta, los mecanismos de comunicación empleados y el flujo de información entre los distintos componentes del sistema. Este análisis permitió identificar los puntos críticos de la infraestructura y determinar las funcionalidades necesarias para mejorar la monitorización y gestión de los dispositivos.

Asimismo, se realizó una revisión de la documentación técnica disponible relacionada con las tecnologías empleadas en el proyecto. Se estudiaron los protocolos MQTT y OPC UA utilizados para las comunicaciones industriales, así como la documentación de las herramientas y *frameworks* seleccionados por la empresa para el desarrollo de la solución. Esta tarea permitió comprender las limitaciones técnicas existentes y garantizar la viabilidad de los requisitos identificados.

Finalmente, también se utilizó la observación directa del funcionamiento de la plataforma y de los procedimientos de mantenimiento existentes. Esta actividad permitió comprender mejor

las tareas realizadas habitualmente por el personal técnico y detectar oportunidades de mejora que posteriormente se tradujeron en requisitos funcionales del sistema.

Como resultado de este proceso se identificaron las principales necesidades del proyecto: disponer de un mecanismo centralizado para la recepción y almacenamiento de métricas de los nodos industriales, facilitar la supervisión del estado de los dispositivos, generar alertas ante situaciones anómalas, gestionar de forma remota determinados elementos de la infraestructura y proporcionar una interfaz web que facilitara la consulta y administración de la información recopilada. Estos hallazgos sirvieron como base para la definición formal de los requisitos funcionales y no funcionales descritos en la siguiente sección.

3.2. Definición de requisitos

Una vez identificadas las necesidades de la empresa durante la fase de investigación, se procedió a la especificación formal de los requisitos del sistema. Para ello se empleó una combinación de casos de uso y descripciones textuales de requisitos, ya que esta aproximación permite representar de forma sencilla las interacciones entre los usuarios y el sistema, así como documentar restricciones funcionales y técnicas que no quedan reflejadas directamente en los diagramas.

La Figura 3.1 muestra el diagrama general de casos de uso del sistema. En él se representan los principales actores identificados durante el análisis: el administrador de la plataforma, los nodos industriales que envían información al sistema, el *backend* del sistema y la base de datos que almacena la información. Después, se muestran los cuadros con las especificaciones de cada caso de uso.

El actor Administrador representa al personal técnico encargado de la supervisión y gestión de la plataforma. Este usuario puede autenticarse, administrar *gateways*, conectores y máquinas, consultar métricas históricas y gestionar las reglas de alerta. Por otro lado, el actor Nodo Edge representa los dispositivos desplegados en planta que envían periódicamente métricas al sistema. A partir de la recepción de estas métricas, el actor *Backend* evalúa las reglas configuradas y genera alertas cuando se detectan situaciones anómalas que llegan al actor Base de datos y se almacenan.

3.2.1. Requisitos funcionales

Los requisitos funcionales describen los servicios que debe proporcionar el sistema para satisfacer las necesidades identificadas durante la fase de análisis. Estos están representados en el Cuadro 3.11.

3.2.2. Requisitos de datos

Debido a que el objetivo principal del sistema consiste en recopilar, almacenar y procesar información procedente de los nodos industriales, se definieron también requisitos específicos

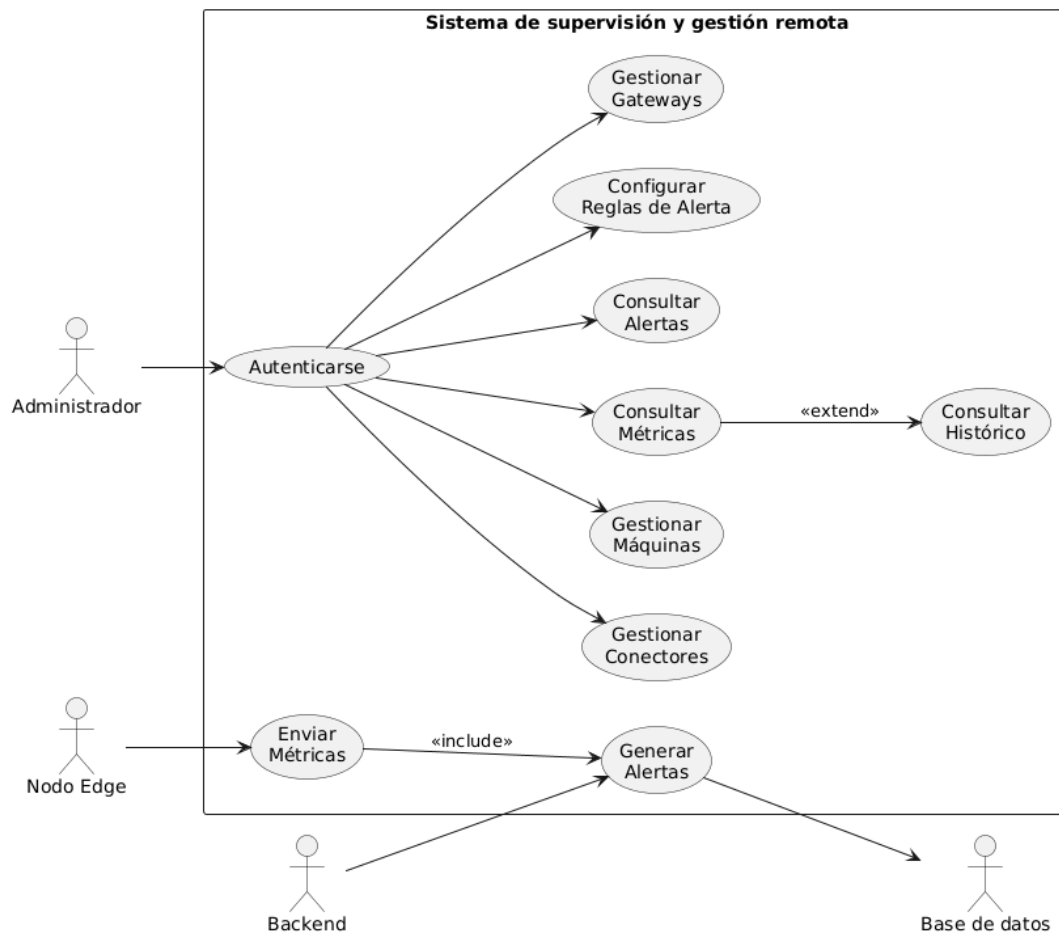


Figura 3.1: Diagrama de casos de uso del sistema

Identificador	CU-01
Nombre	Autenticarse
Actor principal	Administrador
Descripción	Permite al administrador acceder a la plataforma mediante sus credenciales.
Precondiciones	- El usuario debe estar registrado. - La aplicación debe estar disponible.
Postcondiciones	Se inicia una sesión válida que permite acceder a las funcionalidades administrativas del sistema.
Flujo principal	1. El administrador accede a la pantalla de inicio de sesión. 2. Introduce su nombre de usuario y contraseña. 3. El sistema valida las credenciales. 4. El sistema concede el acceso y crea la sesión.
Flujos alternativos	1. Las credenciales son incorrectas y el sistema muestra un mensaje de error. 2. El usuario vuelve a introducir las credenciales.

Cuadro 3.1: Caso de uso CU-01

Identificador	CU-02
Nombre	Gestionar <i>Gateways</i>
Actor principal	Administrador
Descripción	Permite registrar nuevos <i>gateways</i> y modificar, consultar y eliminar <i>gateways</i> ya registrados en el sistema.
Precondiciones	El administrador debe haber iniciado sesión.
Postcondiciones	La información del <i>gateway</i> queda actualizada en la base de datos.
Flujo principal	1. El administrador accede al módulo de <i>gateways</i>. 2. Selecciona la operación deseada. 3. Introduce o modifica los datos. 4. El sistema valida la información. 5. El sistema guarda los cambios.
Flujos alternativos	1. Algún dato es inválido. 2. El sistema informa del error y solicita su corrección.

Cuadro 3.2: Caso de uso CU-02

Identificador	CU-03
Nombre	Gestionar Conectores
Actor principal	Administrador
Descripción	Permite administrar los conectores asociados a cada <i>gateway</i> .
Precondiciones	El administrador ha iniciado sesión.
Postcondiciones	Los conectores quedan registrados o actualizados.
Flujo principal	1. Accede al módulo de conectores. 2. Añade, modifica o elimina un conector. 3. El sistema valida la operación. 4. El sistema guarda los cambios.
Flujos alternativos	1. Algún dato es inválido. 2. El sistema informa del error y solicita su corrección.

Cuadro 3.3: Caso de uso CU-03

Identificador	CU-04
Nombre	Gestionar Máquinas
Actor principal	Administrador
Descripción	Permite registrar y administrar las máquinas monitorizadas.
Precondiciones	El administrador ha iniciado sesión.
Postcondiciones	Las máquinas quedan registradas o actualizadas.
Flujo principal	1. Accede al módulo de máquinas. 2. Selecciona la operación deseada. 3. Introduce los datos. 4. El sistema valida la información. 5. El sistema guarda los cambios.
Flujos alternativos	1. Algún dato es inválido. 2. El sistema informa del error y solicita su corrección.

Cuadro 3.4: Caso de uso CU-04

Identificador	CU-05
Nombre	Configurar Reglas de Alerta
Actor principal	Administrador
Descripción	Permite definir los umbrales y condiciones que originan alertas.
Precondiciones	El administrador ha iniciado sesión y existe una máquina registrada.
Postcondiciones	Las reglas quedan asociadas a la máquina correspondiente.
Flujo principal	1. Selecciona una máquina. 2. Define el atributo, operador, umbral y severidad. 3. El sistema valida los datos. 4. El sistema almacena la regla.
Flujos alternativos	1. Algún dato es inválido. 2. El sistema informa del error y solicita su corrección.

Cuadro 3.5: Caso de uso CU-05

Identificador	CU-06
Nombre	Consultar Alertas
Actor principal	Administrador
Descripción	Permite visualizar las alertas generadas por el sistema.
Precondiciones	El administrador ha iniciado sesión.
Postcondiciones	Se muestran las alertas almacenadas.
Flujo principal	1. Accede al módulo de alertas. 2. El sistema recupera las alertas. 3. El sistema muestra la información.
Flujos alternativos	No existen alertas registradas.

Cuadro 3.6: Caso de uso CU-06

Identificador	CU-07
Nombre	Consultar Métricas
Actor principal	Administrador
Descripción	Permite visualizar las métricas almacenadas de las máquinas monitorizadas.
Precondiciones	El administrador ha iniciado sesión.
Postcondiciones	Las métricas seleccionadas son mostradas al usuario.
Flujo principal	1. El administrador accede al módulo de métricas. 2. Selecciona una máquina. 3. El sistema recupera las métricas. 4. El sistema las muestra.
Flujos alternativos	1. El administrador accede al módulo de métricas. 2. Selecciona una máquina. 3. No existen métricas para la máquina seleccionada.

Cuadro 3.7: Caso de uso CU-07

Identificador	CU-08
Nombre	Consultar Histórico
Actor principal	Administrador
Descripción	Permite consultar el histórico de una métrica concreta. Este caso de uso extiende al caso de uso Consultar Métricas.
Precondiciones	1. El administrador ha iniciado sesión. 2. Se está consultando una métrica.
Postcondiciones	Se muestra la evolución temporal de la métrica seleccionada.
Flujo principal	1. El administrador selecciona una métrica. 2. Solicita consultar su histórico. 3. El sistema recupera los registros históricos. 4. El sistema muestra la evolución temporal.
Flujos alternativos	1. El administrador selecciona una métrica. 2. Solicita consultar su histórico. 3. No existen datos históricos disponibles.

Cuadro 3.8: Caso de uso CU-08

Identificador	CU-09
Nombre	Enviar Métricas
Actor principal	Nodo <i>Edge</i>
Descripción	El nodo industrial envía periódicamente las métricas al servidor mediante servicios REST.
Precondiciones	1. El nodo dispone de conectividad. 2. La máquina está registrada.
Postcondiciones	Las métricas quedan almacenadas y se inicia la evaluación de reglas de alerta.
Flujo principal	1. El nodo recopila las métricas. 2. Envía la petición REST. 3. El sistema valida los datos. 4. El sistema almacena las métricas. 5. Se ejecuta el caso de uso Generar Alertas.
Flujos alternativos	1. El nodo recopila las métricas. 2. Envía la petición REST. 3. La petición contiene datos inválidos o no puede procesarse.

Cuadro 3.9: Caso de uso CU-09

Identificador	CU-10
Nombre	Generar Alertas
Actor principal	<i>Backend</i>
Descripción	El <i>backend</i> evalúa las reglas configuradas y genera una alerta cuando alguna condición se cumple.
Precondiciones	Se han recibido correctamente las métricas de una máquina.
Postcondiciones	Las alertas generadas quedan almacenadas en la base de datos.
Flujo principal	1. El sistema recupera las reglas asociadas a la máquina. 2. Compara cada métrica con sus umbrales. 3. Si alguna condición se cumple, crea una alerta. 4. La alerta queda registrada en la base de datos.
Flujos alternativos	1. El sistema recupera las reglas asociadas a la máquina. 2. Compara cada métrica con sus umbrales. 3. No se incumple ninguna regla, por lo que no se genera ninguna alerta.

Cuadro 3.10: Caso de uso CU-10

relacionados con los datos gestionados por la aplicación. En el Cuadro 3.12 se muestran los requisitos de datos identificados y su relación con los requisitos funcionales.

3.2.3. Requisitos no funcionales

Los requisitos no funcionales describen las propiedades de calidad y las restricciones técnicas que debe cumplir el sistema. Se muestran en el Cuadro 3.13

3.3. Análisis de requisitos

A partir de los requisitos identificados en las secciones anteriores, se realizó un análisis funcional del sistema con el objetivo de determinar los elementos necesarios para proporcionar una solución adecuada al problema planteado. Esta fase permitió identificar las principales entidades del dominio, las relaciones existentes entre ellas y los procesos fundamentales que debía soportar la aplicación.

Durante el análisis se observó que el problema podía dividirse en dos áreas principales. Por un lado, la adquisición y almacenamiento de información procedente de los nodos industriales desplegados en planta. Por otro, la supervisión y administración de dicha información por parte de los usuarios de la plataforma.

Código	Descripción
RF-01	El sistema debe permitir gestionar los <i>gateways</i> registrados en la plataforma. Asociado al requisito P2 del backlog y al caso de uso CU-02
RF-02	El sistema debe permitir gestionar los conectores asociados a cada <i>gateway</i> . Asociado al requisito P2 del backlog y al caso de uso CU-03
RF-03	El sistema debe permitir gestionar las máquinas industriales monitorizadas. Asociado al requisito P2 del backlog y al caso de uso CU-04
RF-04	El sistema debe recibir métricas enviadas por los nodos industriales mediante servicios REST. Asociado a los requisitos P2 y P6 del backlog y al caso de uso CU-09
RF-05	El sistema debe almacenar las métricas recibidas junto con su marca temporal. Asociado a los requisitos P3 y P6 del backlog y al caso de uso CU-07
RF-06	El sistema debe permitir consultar métricas históricas de las máquinas monitorizadas. Asociado a los requisitos P3 y P6 del backlog y a los casos de uso CU-07 y CU-08
RF-07	El sistema debe generar alertas cuando una métrica supere los umbrales configurados. Asociado al requisito P6 del backlog y al caso de uso CU-10
RF-08	El sistema debe permitir consultar las alertas generadas por cada máquina. Asociado al requisito P6 del backlog y al caso de uso CU-06
RF-09	El sistema debe permitir filtrar las alertas según su nivel de severidad. Asociado al requisito P6 del backlog y al caso de uso CU-06
RF-10	El sistema debe proporcionar una interfaz web para la visualización y administración de la información almacenada. Asociado al requisito P4 del backlog y a los casos de uso CU-01 a CU-08
RF-11	El sistema debe permitir lanzar actualizaciones de software remotas. Asociado al requisito P5 del backlog
RF-12	El sistema debe permitir activar y desactivar las actualizaciones remotas para cada nodo. Asociado al requisito P5 del backlog

Cuadro 3.11: Requisitos funcionales del sistema

Código	Descripción
RD-01 (RF-01, RF-02, RF-03)	El sistema debe almacenar información de <i>gateways</i> , conectores y máquinas industriales como código de identificación, nombre, nodos relacionados, ubicación, modelo y número de serie, entre otros.
RD-02 (RF-04, RF-05)	El sistema debe almacenar información sobre las métricas. Cada métrica almacenada debe incluir un identificador del dispositivo origen, un indicador de unidad medida, un valor medido y una marca temporal.
RD-03 (RF-04, RF-05, RF-06)	El sistema debe conservar el histórico de métricas para permitir consultas posteriores. Esto significa poder consultar por marca temporal.
RD-04 (RF-07, RF-08, RF-09)	El sistema debe almacenar información sobre las alertas que se lancen. Las alertas generadas deben almacenar un indicador de unidades, un valor de umbral superado y un valor de la métrica vigilada que haya superado el umbral junto con su fecha de creación, severidad y origen.
RD-05 (RF-01, RF-02, RF-03)	El sistema debe mantener la integridad de las relaciones entre máquinas, conectores y <i>gateways</i> . Esto implica almacenar la jerarquía de nodos: En las <i>gateways</i> se debe almacenar los conectores dependientes de las <i>gateways</i> , en los conectores se debe almacenar las máquinas dependientes del conector y la <i>gateway</i> padre del conector y en las máquinas se debe almacenar el conector padre de la máquina.

Cuadro 3.12: Requisitos de datos

Código	Descripción
RNF-01	El sistema debe estar disponible de forma continua para la recepción de métricas.
RNF-02	La arquitectura debe permitir incorporar nuevos dispositivos sin modificaciones significativas del software.
RNF-03	La aplicación debe desplegarse mediante contenedores Docker.
RNF-04	El sistema debe organizarse siguiendo una arquitectura modular que facilite el mantenimiento y la evolución futura.
RNF-05	Las operaciones de consulta deben responder en tiempos adecuados para permitir una monitorización eficaz.
RNF-06	El sistema debe garantizar la integridad de los datos almacenados.
RNF-07	La interfaz web debe ser accesible desde los navegadores modernos utilizados por la empresa.
RNF-08	El sistema debe ser compatible con la infraestructura tecnológica existente en la organización.

Cuadro 3.13: Requisitos no funcionales

3.3.1. Modelo conceptual

El análisis del dominio permitió identificar las entidades principales del sistema y sus relaciones. La Figura 3.2 muestra el modelo conceptual obtenido durante esta fase.

Las entidades identificadas fueron las siguientes:

- **Gateway:** representa un nodo industrial desplegado en planta encargado de recopilar información y comunicarse con el servidor central.
- **Conector:** elemento encargado de establecer la comunicación entre un *gateway* y las máquinas industriales utilizando los protocolos soportados.
- **Máquina:** representa un dispositivo o equipo industrial cuyos datos son monitorizados por el sistema.
- **Métrica:** valor obtenido periódicamente de una máquina o dispositivo, acompañado de una marca temporal.
- **Alerta:** incidencia generada cuando una métrica incumple una condición previamente definida.
- **Regla de alerta:** conjunto de condiciones utilizadas para evaluar las métricas recibidas y determinar si debe generarse una alerta.
- **Usuario:** persona autorizada para acceder a la plataforma y administrar los recursos disponibles.

El análisis de estas entidades permitió comprobar que el sistema podía estructurarse alrededor de un modelo centralizado donde los *gateways* envían información al servidor y éste se encarga de almacenarla, procesarla y ponerla a disposición de los usuarios.

3.3.2. Análisis de los procesos principales

Tras estudiar los requisitos funcionales, se identificaron tres procesos fundamentales dentro del sistema. En el Cuadro 3.3 se muestra una representación del diagrama de flujo de alertas del sistema.

Recepción y almacenamiento de métricas

El primer proceso consiste en la recepción de métricas procedentes de los nodos industriales. Cuando un *gateway* obtiene información de una máquina industrial, envía los datos al servidor mediante la API REST. El servidor valida la información recibida, la almacena en la base de datos y la deja disponible para futuras consultas.

Este proceso responde a los requisitos relacionados con la monitorización continua y el almacenamiento histórico de información.

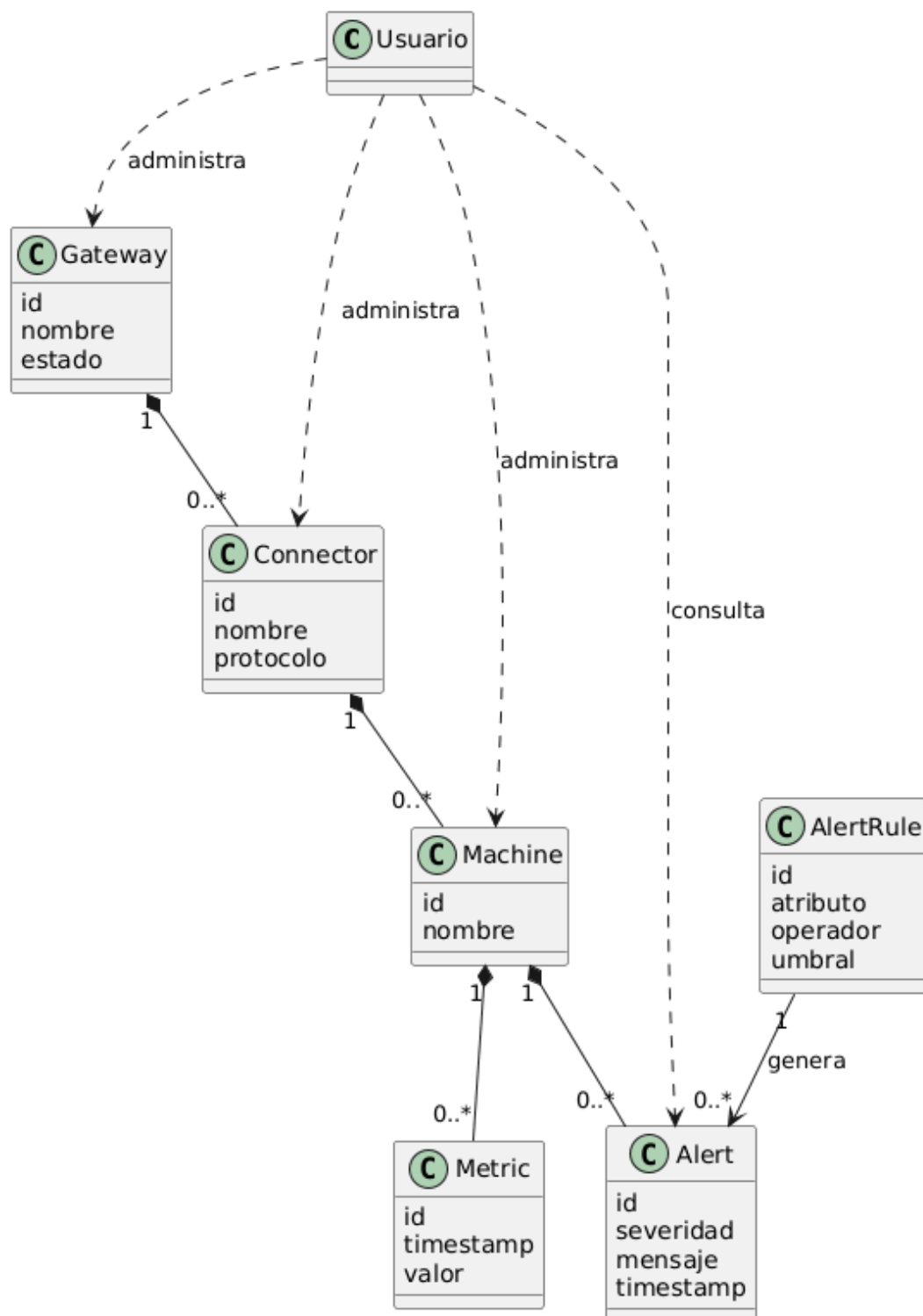


Figura 3.2: Modelo conceptual del sistema

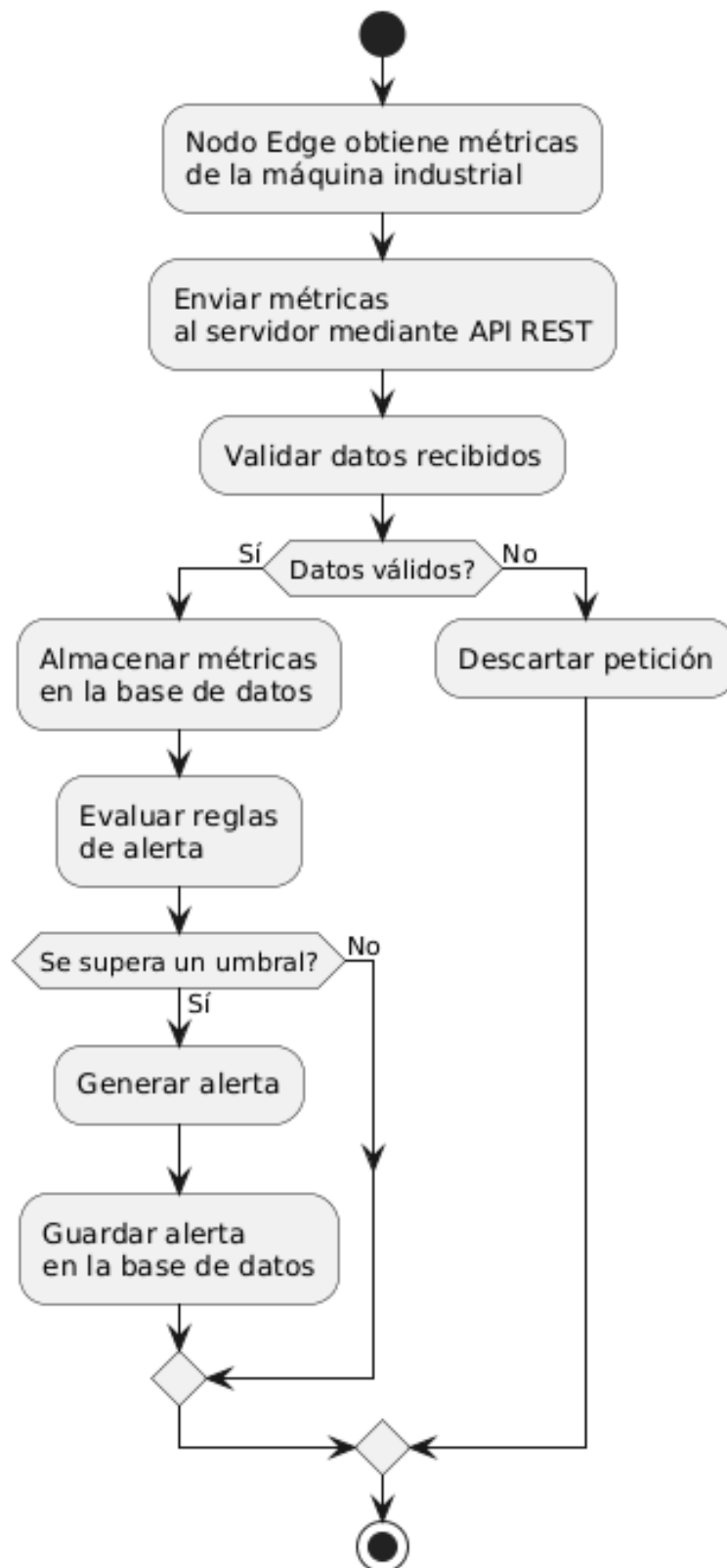


Figura 3.3: Diagrama de flujo de alertas del sistema

Evaluación de alertas

Una vez almacenadas las métricas, el sistema debe comprobar si los valores recibidos cumplen las reglas de alerta configuradas. Para ello se analizan las condiciones asociadas a cada métrica y se generan alertas cuando se detectan situaciones anómalas.

Este mecanismo permite detectar incidencias de forma automática sin necesidad de supervisión constante por parte de los usuarios.

Supervisión y administración

El tercer proceso corresponde a la interacción de los usuarios con la plataforma web. Los administradores pueden consultar el estado de los nodos, visualizar métricas históricas, revisar alertas y gestionar los elementos configurados en el sistema.

Esta funcionalidad proporciona una visión centralizada de toda la infraestructura monitorizada y facilita las tareas de mantenimiento.

3.3.3. Análisis de alternativas

Durante el análisis se consideraron diferentes alternativas para la arquitectura general de la solución.

La primera alternativa consistía en que cada nodo almacenara localmente la información recopilada y proporcionara mecanismos independientes de consulta. Esta opción fue descartada debido a la dificultad para centralizar la supervisión y realizar análisis globales sobre todos los dispositivos desplegados.

La segunda alternativa consistía en utilizar una arquitectura centralizada donde todos los nodos enviaran la información a un servidor común. Esta solución simplifica la gestión de la infraestructura, facilita el almacenamiento histórico de datos y permite implementar mecanismos globales de alerta y supervisión.

Debido a estas ventajas, se seleccionó la arquitectura centralizada como base para el desarrollo del sistema.

3.3.4. Resultados del análisis

Como resultado de esta fase se determinó que la solución debía estar compuesta por los siguientes subsistemas:

- Un módulo de recepción de métricas mediante servicios REST.

- Un módulo de persistencia encargado del almacenamiento de datos.
- Un módulo de evaluación y gestión de alertas.
- Un módulo de administración de *gateways*, conectores y máquinas.
- Una interfaz web para la supervisión y gestión de la plataforma.

Las conclusiones obtenidas durante el análisis sirvieron como base para el diseño detallado de la arquitectura, la base de datos y los distintos componentes software descritos en el capítulo siguiente.

Capítulo 4

Diseño

4.1. Diseño de la base de datos

La persistencia de la información constituye uno de los componentes fundamentales del sistema desarrollado, ya que la plataforma debe almacenar tanto la configuración de la infraestructura monitorizada como el histórico de métricas y alertas generadas durante su funcionamiento.

Debido a la naturaleza estructurada de la información y a la necesidad de garantizar la integridad de los datos, se optó por utilizar una base de datos relacional. Durante esta fase se transformó el modelo conceptual obtenido en el análisis en un modelo lógico compuesto por un conjunto de tablas relacionadas mediante claves primarias y claves foráneas.

La Figura 4.1 muestra el modelo lógico diseñado para la base de datos del sistema.

4.1.1. Entidad *Gateway*

La entidad *Gateway* representa los nodos edge desplegados en planta. Cada *gateway* almacena una descripción identificativa y actúa como elemento principal de la infraestructura monitorizada.

Un *gateway* puede tener asociados múltiples conectores y múltiples máquinas industriales.

4.1.2. Entidad *Connector*

La entidad *Connector* representa los mecanismos de comunicación utilizados para acceder a las máquinas industriales. Cada conector dispone de un nombre identificativo y pertenece a un único *gateway*.

La relación entre *gateways* y conectores es de uno a muchos, ya que un *gateway* puede

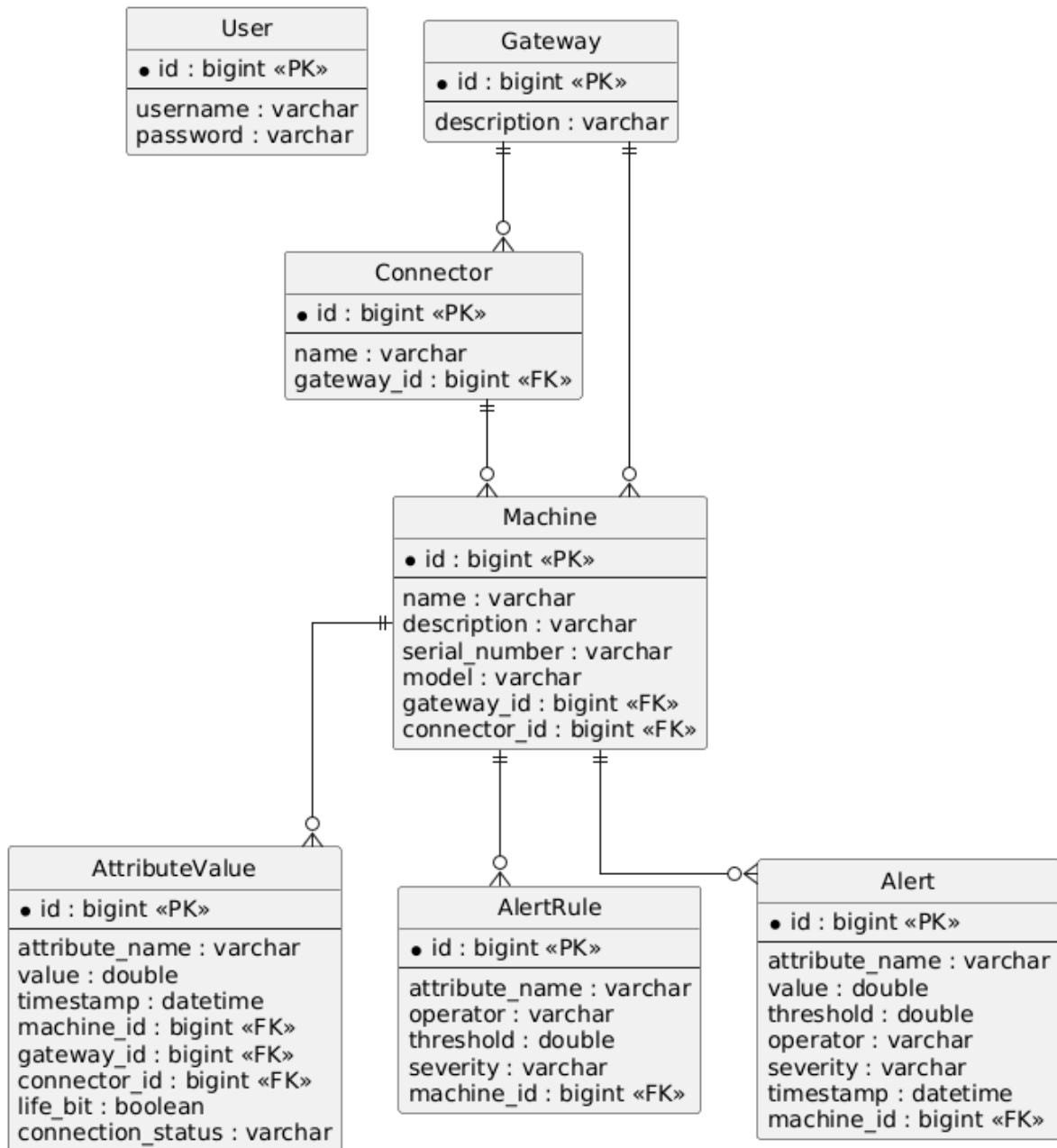


Figura 4.1: Modelo lógico (entidad-relación) de la base de datos

disponer de varios conectores simultáneamente.

4.1.3. Entidad Machine

La entidad **Machine** almacena la información básica de cada máquina industrial monitorizada por el sistema.

Para cada máquina se registran atributos descriptivos como el nombre, la descripción, el número de serie y el modelo. Además, cada máquina se asocia a un *gateway* y a un conector, permitiendo conocer tanto el nodo desde el que se monitoriza como el mecanismo de comunicación utilizado.

Esta entidad constituye el elemento central del modelo de datos, ya que tanto las métricas como las alertas se encuentran vinculadas a ella.

4.1.4. Entidad AttributeValue

La entidad **AttributeValue** almacena los valores obtenidos durante el proceso de monitorización. Cada registro representa una medición individual asociada a un atributo concreto de una máquina.

Además del valor capturado, se almacena la fecha y hora de adquisición, así como los identificadores del *gateway*, conector y máquina que originaron la información.

También se registran campos auxiliares relacionados con el estado operativo del dispositivo, como el indicador **lifeBit** y el estado de conexión.

Debido al elevado volumen de datos esperado en esta tabla, se definió un índice compuesto formado por los campos **machineId**, **attributeName** y **timestamp**. Este índice permite optimizar las consultas históricas de métricas, que constituyen una de las operaciones más frecuentes de la aplicación.

4.1.5. Entidad AlertRule

La entidad **AlertRule** almacena las reglas utilizadas para detectar situaciones anómalas durante la monitorización.

Cada regla define el atributo que debe evaluarse, el operador de comparación utilizado, el valor umbral y el nivel de severidad asociado a la alerta generada. Las reglas se encuentran vinculadas a una máquina específica, permitiendo configurar criterios de supervisión personalizados para cada dispositivo.

4.1.6. Entidad **Alert**

La entidad **Alert** almacena las incidencias detectadas por el sistema cuando una métrica incumple alguna de las reglas configuradas.

Cada alerta registra el atributo afectado, el valor detectado, el umbral configurado, el operador utilizado para la comparación, el nivel de severidad y la fecha de generación. Además, mantiene una referencia a la máquina que originó la incidencia.

4.1.7. Entidad **User**

La entidad **User** almacena la información necesaria para la autenticación de usuarios dentro de la plataforma.

Para cada usuario se registra un nombre de usuario único y la contraseña asociada, permitiendo controlar el acceso a las funcionalidades administrativas del sistema.

4.2. Diseño de software

Una vez definido el modelo de datos, se procedió al diseño de la arquitectura software del sistema. El objetivo de esta fase fue estructurar la aplicación de forma modular para facilitar su mantenimiento, evolución y escalabilidad, garantizando además una separación clara de responsabilidades entre los distintos componentes.

Para ello se adoptó una arquitectura basada en capas siguiendo las recomendaciones habituales para aplicaciones web desarrolladas con NestJS. Esta arquitectura permite desacoplar la lógica de negocio, el acceso a datos y la exposición de servicios mediante API REST.

4.2.1. Arquitectura general

La aplicación se organiza en varios módulos funcionales independientes, cada uno de ellos responsable de una parte concreta del dominio del problema. Entre los principales módulos se encuentran:

- Gestión de *gateways*.
- Gestión de conectores.
- Gestión de máquinas.
- Gestión de métricas.
- Gestión de alertas.

- Gestión de reglas de alerta.
- Autenticación y gestión de usuarios.

Cada módulo sigue una estructura común basada en controladores, servicios y entidades, favoreciendo la cohesión interna y reduciendo el acoplamiento entre componentes.

4.2.2. Patrón arquitectónico

La arquitectura implementada puede considerarse una adaptación del patrón MVC para aplicaciones basadas en servicios REST. El Cuadro 4.2 muestra cómo se ha adaptado el patrón MVC a la aplicación.

- Los **controladores** actúan como punto de entrada a la aplicación, recibiendo las peticiones HTTP y devolviendo las respuestas correspondientes.
- Los **servicios** contienen la lógica de negocio del sistema y coordinan las operaciones necesarias para procesar cada petición.
- Las **entidades** y repositorios gestionan la persistencia de la información mediante el uso de TypeORM.

Esta organización permite mantener una clara separación entre presentación, lógica de negocio y persistencia de datos, facilitando la modificación de una de las tres partes sin afectar a las demás. Esta característica facilita la evolución futura del sistema y mejora su mantenibilidad.

Asimismo, el uso de una arquitectura modular junto con la inyección de dependencias proporcionada por NestJS favorece la escalabilidad de la aplicación y simplifica la incorporación de nuevas funcionalidades.

4.2.3. Inyección de dependencias

NestJS incorpora de forma nativa un mecanismo de inyección de dependencias que fue utilizado en todo el proyecto.

Gracias a este mecanismo, los controladores reciben automáticamente las instancias de los servicios necesarios y los servicios obtienen acceso a los repositorios de persistencia sin necesidad de crear manualmente las dependencias.

Esta aproximación reduce el acoplamiento entre componentes y facilita tanto las pruebas como el mantenimiento del sistema.

Patrón arquitectónico MVC adaptado para servicios REST

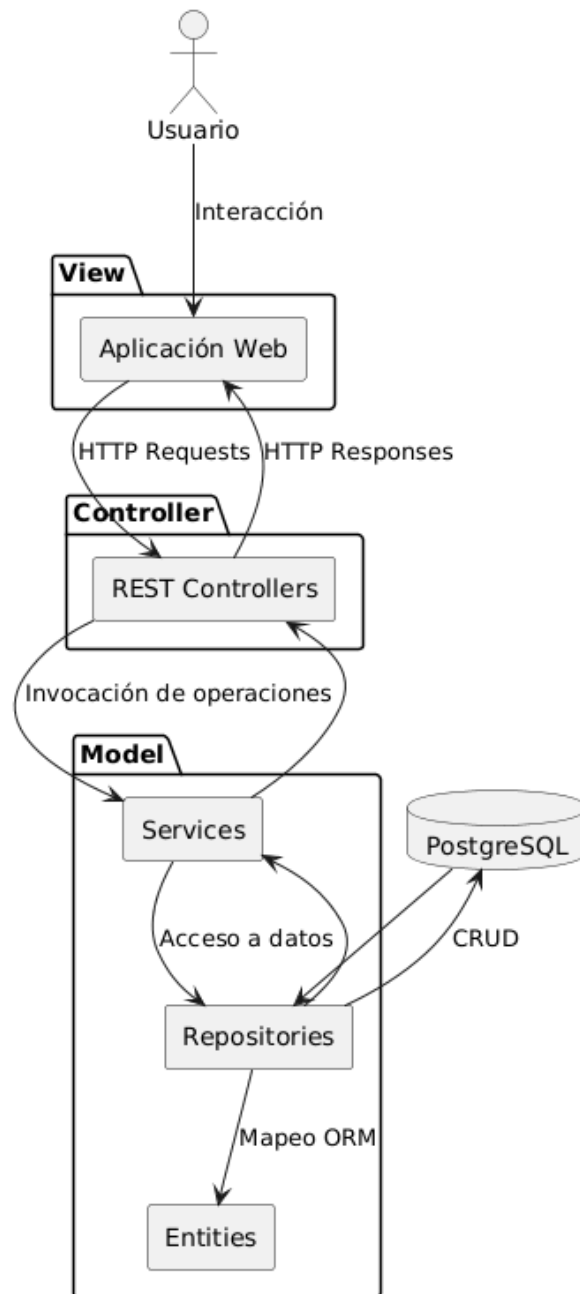


Figura 4.2: Arquitectura MVC de la aplicación

4.2.4. Diseño modular

La aplicación se diseñó siguiendo una estructura modular donde cada módulo encapsula una funcionalidad específica.

Esta decisión proporciona diversas ventajas:

- Facilita la comprensión del código.
- Reduce las dependencias entre funcionalidades.
- Permite incorporar nuevas características con un impacto reducido sobre el resto del sistema.
- Favorece la reutilización de componentes.

Además, esta organización se adapta especialmente bien al crecimiento futuro de la plataforma, permitiendo añadir nuevos tipos de dispositivos o mecanismos de monitorización sin alterar significativamente la estructura existente.

4.2.5. Flujo principal de monitorización

El proceso más importante del sistema corresponde a la recepción y procesamiento de métricas procedentes de los nodos industriales.

Cuando un *gateway* envía información al servidor, la petición es recibida por el controlador correspondiente. Posteriormente, el servicio encargado valida los datos recibidos, almacena las métricas en la base de datos y ejecuta el proceso de evaluación de reglas de alerta.

Si alguna de las condiciones configuradas se incumple, el sistema genera una nueva alerta y la almacena para su posterior consulta desde la interfaz web.

La Figura 4.3 muestra el diagrama de secuencia asociado a este proceso.

4.3. Diseño de las interfaces

La interfaz de usuario se diseñó con el objetivo de proporcionar una herramienta sencilla e intuitiva para la supervisión y gestión de la infraestructura industrial monitorizada. Durante el diseño se priorizaron criterios de usabilidad, claridad visual y facilidad de acceso a la información más relevante para los usuarios de la plataforma.

La aplicación web constituye el principal punto de interacción entre los usuarios y el sistema, permitiendo gestionar los distintos elementos de la infraestructura y consultar el estado de las máquinas monitorizadas en tiempo real.

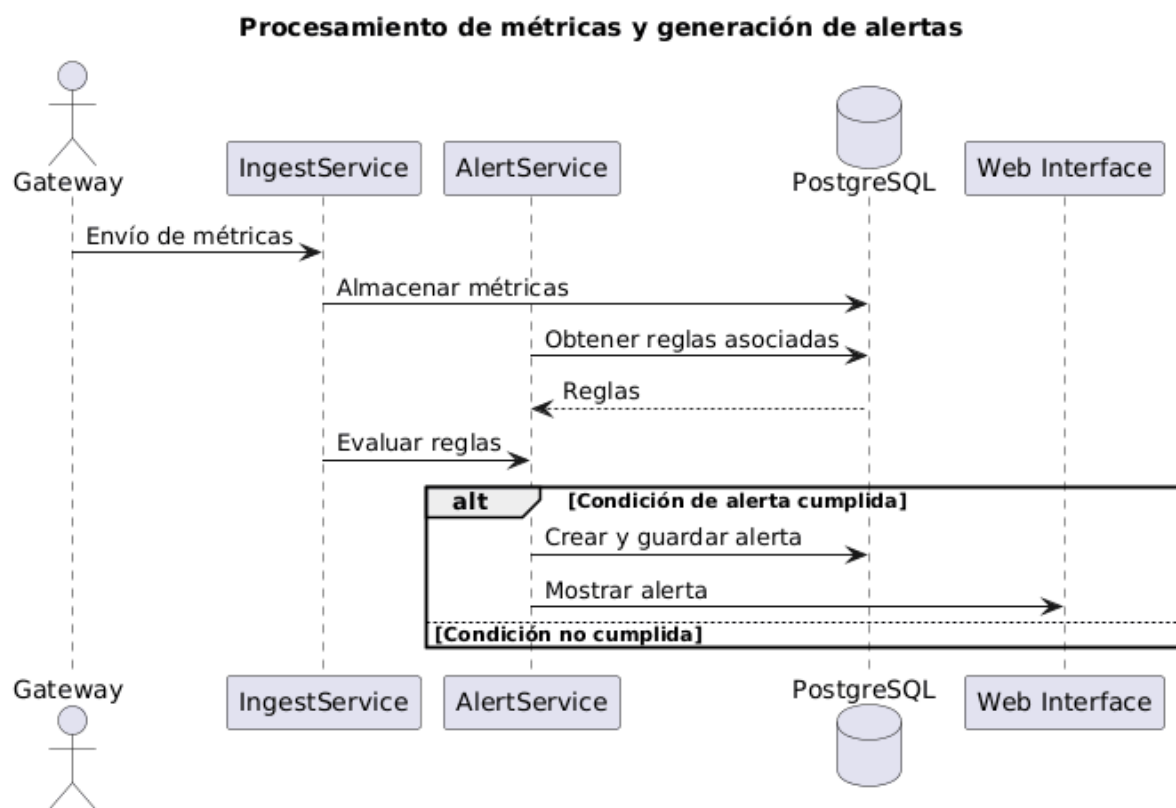


Figura 4.3: Diagrama de secuencia del flujo principal de la aplicación



Figura 4.4: Paleta de colores de las interfaces

4.3.1. Guia de estilo visual

La interfaz gráfica de usuario emplea el tema visual proporcionado por defecto por el framework Ionic, sin aplicar una personalización específica de colores o tipografías. De esta forma, la aplicación utiliza la tipografía Roboto, una tipografía *sans-serif* limpia que garantiza una apariencia formal y una correcta legibilidad.

Respecto a la paleta cromática, se emplean los colores predeterminados de Ionic, basados en Material Design. En la Figura 4.4 se pueden visualizar los colores de la interfaz. El fondo de las páginas corresponde a un blanco neutro (#f4f5f8) que asegura la legibilidad en entornos con poca y mucha luz, el color principal de la aplicación corresponde al azul (#3880ff), utilizado en botones y elementos destacados de la interfaz, mientras que el resto de acciones emplean los colores semánticos proporcionados por el framework: verde (#2dd36f) para mensajes de éxito o métricas que no superan los umbrales, amarillo(#ffc409) para advertencias y métricas cercanas al umbral y rojo (#eb445a) para acciones de eliminación o mensajes de error, así como para las alertas generadas por métricas que superan su umbral definido. Esta elección permite ofrecer una interfaz consistente, moderna y fácilmente reconocible para el usuario, sin problemas de accesibilidad.

4.3.2. Pantallas principales

Gestión de máquinas

La interfaz de gestión de máquinas permite consultar los dispositivos registrados en la plataforma, crear nuevos elementos y modificar su configuración.

Cada máquina muestra información relevante para su identificación y supervisión, facilitando las tareas de administración de la infraestructura monitorizada.

Visualización de métricas

La visualización de métricas constituye una de las funcionalidades principales del sistema. Desde esta pantalla el usuario puede consultar los valores recibidos por los dispositivos monitorizados y analizar su evolución temporal.

La información se presenta mediante tablas que facilitan la interpretación del estado de las máquinas.

Gestión de alertas

La sección de alertas permite consultar las incidencias detectadas automáticamente por el sistema a partir de las reglas configuradas.

Las alertas muestran información sobre la máquina afectada, la condición que ha provocado la incidencia y el instante en que fue generada.

Configuración de reglas de alerta

La interfaz de configuración de reglas permite definir los umbrales y condiciones que determinarán la generación automática de alertas.

Gracias a esta funcionalidad, el comportamiento del sistema puede adaptarse a diferentes tipos de dispositivos y escenarios de monitorización sin necesidad de modificar el código fuente.

4.4. Diseño de la arquitectura

La arquitectura del sistema define la distribución física de los componentes software y hardware que intervienen en la solución desarrollada, así como los mecanismos de comunicación utilizados entre ellos.

El objetivo principal de esta arquitectura es permitir la recopilación centralizada de información procedente de dispositivos industriales desplegados en planta, su almacenamiento persistente y su posterior consulta a través de una interfaz web.

4.4.1. Arquitectura general

La solución desarrollada sigue una arquitectura cliente-servidor compuesta por cuatro elementos principales:

- Los nodos *Gateway Edge*, encargados de recopilar información de las máquinas industriales.
- El servidor de aplicaciones, donde se ejecuta el *backend* desarrollado con NestJS y se implementa la lógica de negocio del sistema.
- El sistema de persistencia basado en PostgreSQL, encargado de almacenar tanto la configuración de la infraestructura como las métricas y alertas generadas.
- Los clientes web utilizados por los usuarios para consultar y administrar la información almacenada.

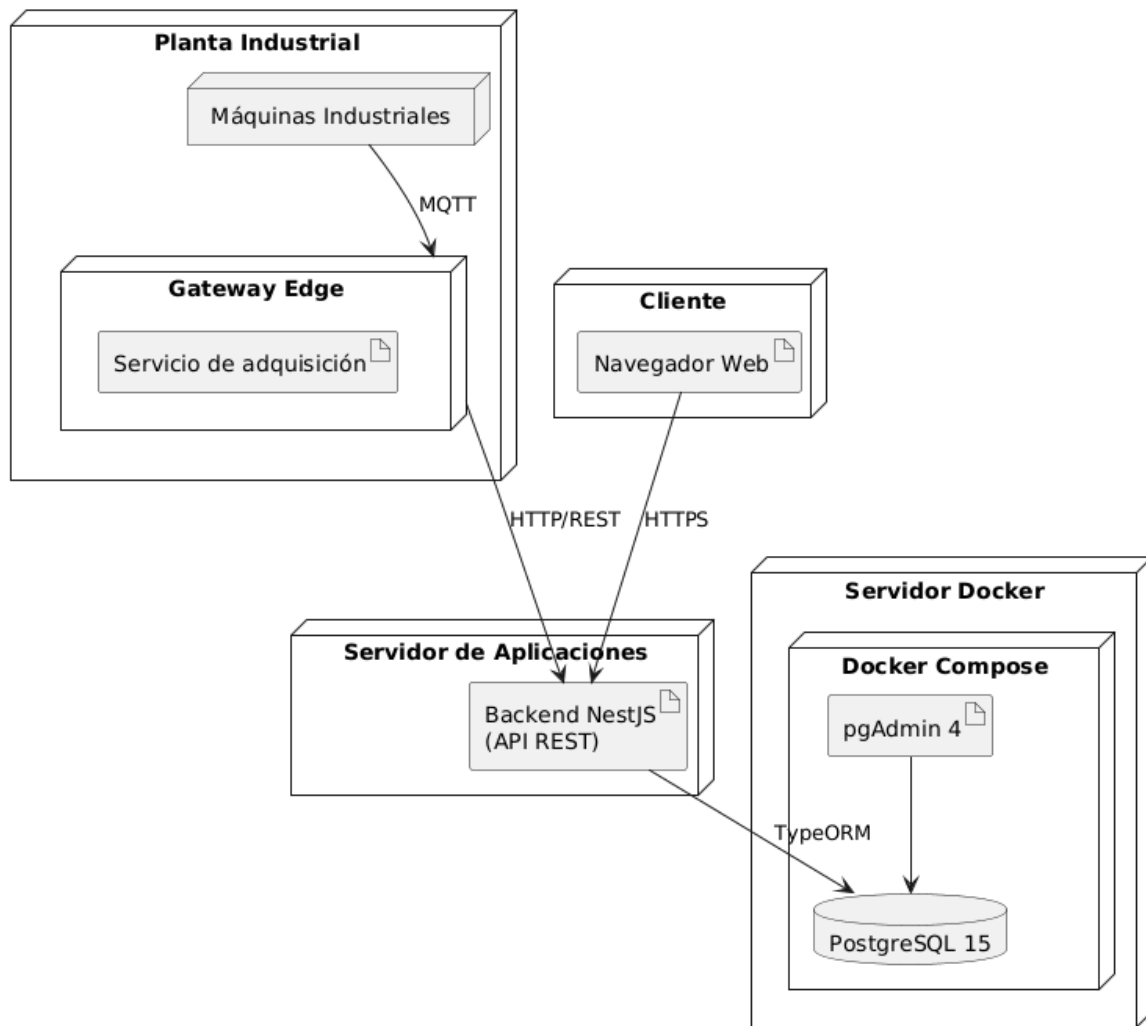


Figura 4.5: Diagrama de despliegue del sistema

Esta arquitectura permite desacoplar la adquisición de datos, el procesamiento de la información y la visualización de resultados, facilitando la escalabilidad y el mantenimiento de la plataforma.

La Figura 4.5 muestra el diagrama de arquitectura definido para el sistema.

Los *gateways* desplegados en planta obtienen información de las máquinas industriales mediante los protocolos de comunicación soportados por la infraestructura. Posteriormente, los datos recopilados son enviados al servidor central mediante peticiones HTTP a la API REST desarrollada.

El *backend* procesa la información recibida, evalúa las reglas de alerta configuradas y almacena los resultados en la base de datos PostgreSQL.

Los usuarios acceden a la plataforma mediante un navegador web que consume los servicios proporcionados por la API para consultar métricas, visualizar alertas y administrar los

dispositivos monitorizados.

4.4.2. Infraestructura de persistencia

Para la gestión de datos se utilizó PostgreSQL 15 como sistema gestor de bases de datos relacional.

La base de datos se despliega mediante contenedores Docker utilizando Docker Compose, lo que simplifica la configuración del entorno y facilita la reproducibilidad del despliegue.

Además, se incorporó pgAdmin como herramienta de administración, permitiendo realizar tareas de supervisión, mantenimiento y consulta de la base de datos durante el desarrollo y las pruebas del sistema.

La persistencia de los datos se garantiza mediante el uso de volúmenes Docker, evitando la pérdida de información cuando los contenedores son reiniciados o recreados.

4.4.3. Justificación de la arquitectura

La arquitectura seleccionada proporciona una separación clara entre adquisición de datos, lógica de negocio y persistencia. Esta distribución permite escalar cada componente de forma independiente y simplifica el mantenimiento de la aplicación.

Asimismo, el uso de una base de datos centralizada facilita la consulta histórica de métricas, la gestión de alertas y la supervisión global de toda la infraestructura monitorizada, aspectos fundamentales para los objetivos del proyecto.

Capítulo 5

Implementación y pruebas

5.1. Estructura del código

Este capítulo describe la organización del código fuente desarrollado durante el proyecto. La estructura adoptada sigue las recomendaciones del *framework* NestJS y se basa en una organización modular que favorece la mantenibilidad, la reutilización de componentes y la separación de responsabilidades.

5.1.1. Organización general

La aplicación *backend* se encuentra estructurada en módulos funcionales independientes, cada uno de ellos encargado de gestionar una parte específica del dominio del problema. Esta organización permite encapsular la lógica relacionada con cada funcionalidad y reduce el acoplamiento entre componentes. El Cuadro 5.1 muestra una captura con la estructura de directorios del *backend* de la aplicación.

De forma general, el *backend* del proyecto se divide en los siguientes módulos principales:

- Gestión de *gateways*.
- Gestión de conectores.
- Gestión de máquinas.
- Gestión de métricas.
- Gestión de alertas.
- Gestión de reglas de alerta.
- Gestión de usuarios y autenticación.

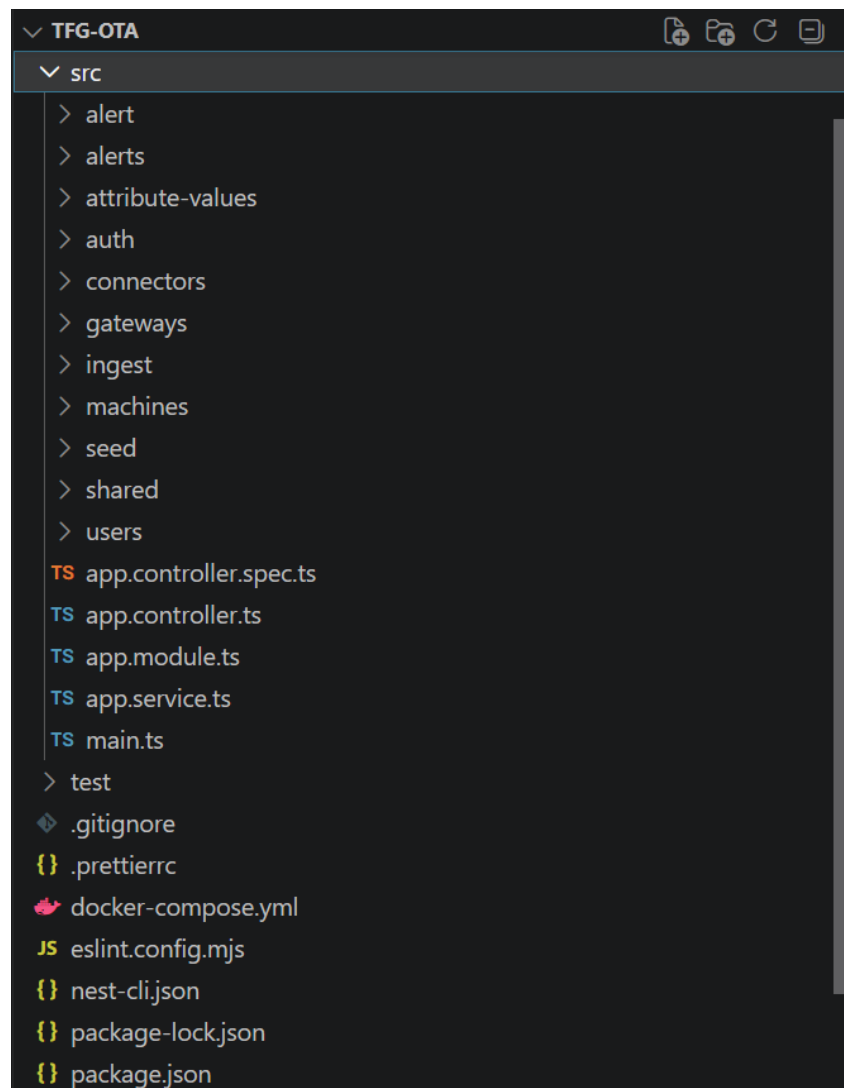


Figura 5.1: Captura de la estructura de directorios del proyecto

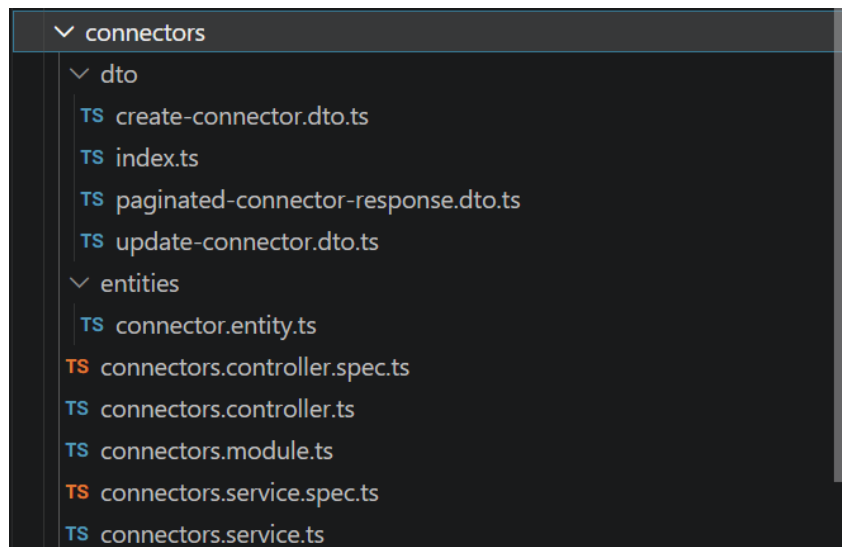


Figura 5.2: Captura de la organización de uno de los módulos del proyecto

Aunque, como se puede ver en el Cuadro 5.1, también se incluyen módulos auxiliares o con otras funcionalidades como:

- Ingesta de métricas.
- Seeding de la base de datos.
- Módulo de métodos compartidos.
- Módulo de tests.

Cada módulo sigue una estructura homogénea basada en controladores, servicios y entidades, además de data transfer objects (dto) y ficheros de testing (*.spec.ts). Se puede comprobar en el Cuadro 5.2 la organización de uno de los módulos.

En el *frontend* se separan las páginas, los modelos y los servicios de acceso. Las rutas de acceso a *gateways*, máquinas y autenticación se cargan de manera diferida. El token obtenido al iniciar sesión se conserva en el almacenamiento local del navegador y el guardia de rutas evita el acceso a las pantallas protegidas cuando no está presente. Angular e Ionic se emplean como base del cliente.

El Cuadro 5.3 muestra una captura con la estructura de directorios del *frontend* de la aplicación.

5.1.2. Controladores

Los controladores constituyen el punto de entrada de las peticiones HTTP recibidas por la aplicación. Su función principal consiste en recibir las solicitudes HTTP de los clientes y llamar a

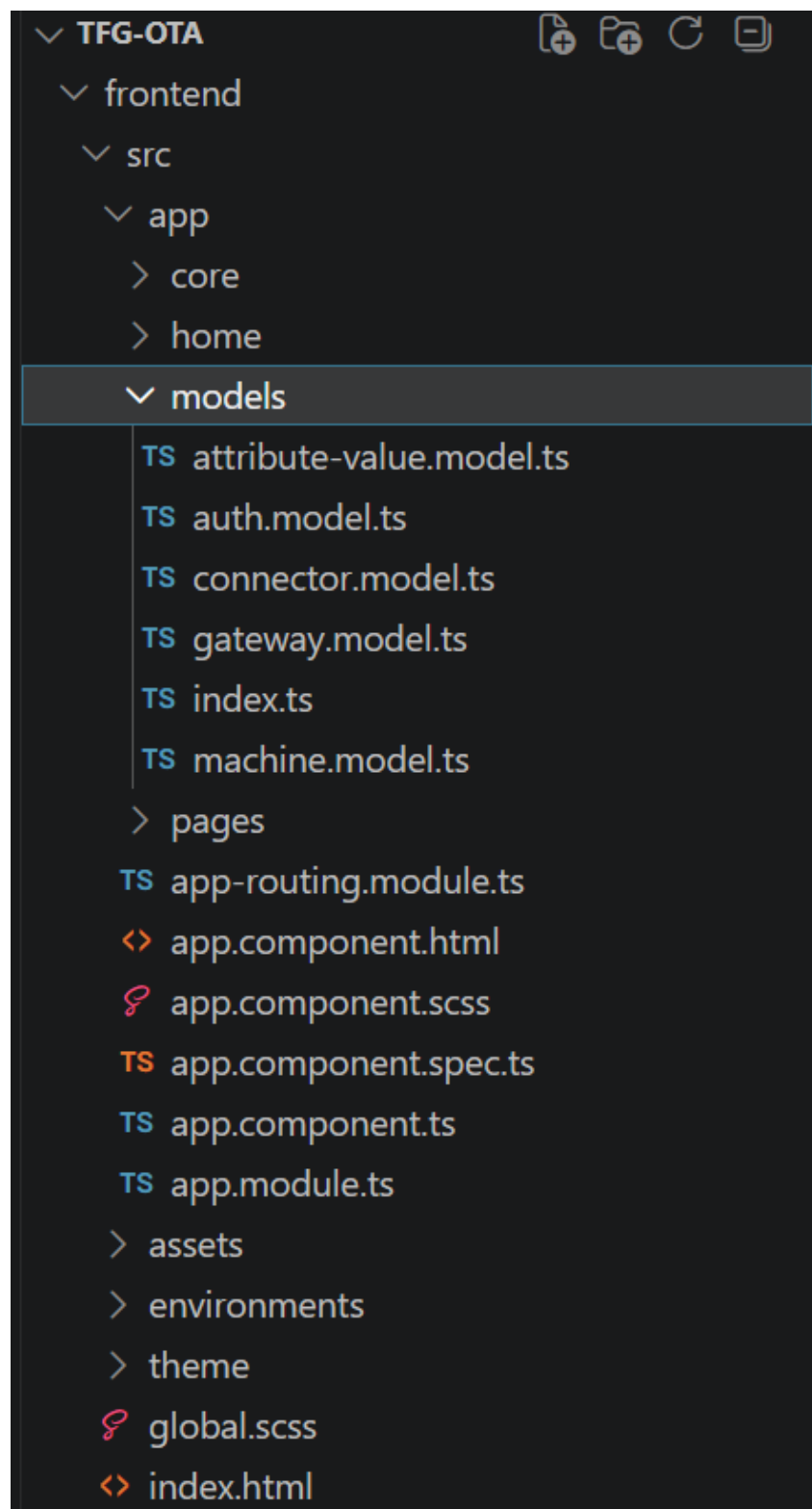


Figura 5.3: Captura de la estructura de directorios del *frontend* del proyecto

los servicios correspondientes para que lleven a cabo la validación y el procesamiento de la información. Cada módulo dispone de su propio controlador, nombrado `nombreMódulo.controller.ts`, lo que permite mantener separadas las distintas funcionalidades de la plataforma.

El Listado 5.4 muestra un ejemplo de controlador.

```
@Controller('machines')
export class MachinesController {
  constructor(private readonly machinesService: MachinesService) {}

  @Post()
  create(@Body() createMachineDto: CreateMachineDto) {
    return this.machinesService.create(createMachineDto);
  }

  @Get()
  findAll(@Query() paginationDto: SearchDto) {
    return this.machinesService.findAll(paginationDto);
  }
}
```

Listado 5.4. Fragmento de `MachinesController`.

5.1.3. Servicios

Los servicios contienen la lógica de negocio de la aplicación. Son responsables de implementar las operaciones necesarias para gestionar los distintos recursos del sistema, coordinar el acceso a la base de datos y aplicar las reglas definidas durante la fase de análisis. Cada módulo tiene su servicio nombrado `nombreMódulo.service.ts`. Esta separación entre controladores y servicios facilita la reutilización del código y mejora la mantenibilidad de la aplicación.

El Listado 5.5 muestra un ejemplo de servicio.

```
async ingest(dto: IngestHierarchyDto) {
  const attributeValues = dto.attributes.map((attr) =>
    this.attributeValueRepository.create({
      [...]
    })),
  );

  await this.attributeValueRepository.save(attributeValues);
  for (const attr of dto.attributes) {
    await this.alertsService.evaluate(
      [...]
    );
  }

  return {
    status: 'ok',
  };
}
```

```

        inserted: attributeValues.length,
    };
}

```

Listado 5.5. Fragmento de `IngestService`.

5.1.4. Entidades

Las entidades representan el modelo de datos utilizado por la aplicación y establecen la correspondencia entre los objetos del sistema y las tablas de la base de datos mediante TypeORM. Son los archivos nombrados `nombreEntidad.entity.ts`.

Además de definir los atributos de cada elemento, las entidades incluyen las relaciones necesarias para representar la estructura de la infraestructura monitorizada.

El Listado 5.6 muestra un ejemplo de entidad.

```

@Entity()
export class Connector {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    name: string;

    @ManyToOne(() => Gateway, (gateway) => gateway.connectors, {
        onDelete: 'CASCADE',
    })
    gateway: Gateway;

    @OneToMany(() => Machine, (machine) => machine.connector)
    machines: Machine[];
}

```

Listado 5.6. Fragmento de `ConnectorEntity`.

5.1.5. DTO: Data Transfer Objects

Se trata de los archivos con nombre `nombreArchivo.dto.ts`. Estos archivos incluyen una o varias clases exportadas con las que se definen la estructura y el tipo de datos que se transfieren entre servicios, controladores o módulos. Están compuestos por una o varias clases y sus respectivas definiciones, con pares clave: tipo; para cada atributo de la clase.

Sirven para unificar formatos entre las distintas funciones que necesiten usarlos, de forma que la función siempre recibirá exactamente los datos que están en el DTO.

El Listado 5.7 muestra un DTO sencillo que se emplea para transferir la información al método del servicio que crea una máquina en la base de datos.

```
export class CreateMachineDto {
  description: string;
  serial: string;
  model: string;
  connectorId: number;
  gatewayId: number;
}
```

Listado 5.7. Fragmento de `CreateMachineDto`.

5.1.6. Interacción entre componentes

La Figura 4.3 muestra el diseño de alto nivel. Su realización concreta se resume en la siguiente secuencia:

1. Un nodo *edge*, o el simulador utilizado en las pruebas, envía una métrica o paquete de métricas al endpoint de ingesta.
2. El controlador recibe el DTO y el servicio comprueba que existen el *gateway*, el conector y la máquina, y que están enlazados entre sí.
3. Para cada atributo se crea una entidad `AttributeValue` con su valor, fecha, origen, `lifeBit` y estado de conexión. El paquete se guarda en PostgreSQL mediante un único `save`.
4. Después de persistir los valores, `IngestService` invoca `AlertsService.evaluate` para cada atributo recibido.
5. El servicio de alertas recupera las reglas de la máquina y atributo, evalúa el operador frente al umbral y, si se cumple la condición, crea y guarda una `Alert`.
6. Por último, `AlertsGateway` emite el evento `alert` a los clientes WebSocket conectados. Las alertas también quedan disponibles para consulta REST.

El Listado 5.8 muestra el paso 4 de esta coordinación. Se ha abreviado únicamente la creación de las entidades para resaltar la llamada a `AlertsService.evaluate`, con todos los datos de cada métrica recibida.

```
await this.attributeValueRepository.save(attributeValues);

for (const attr of dto.attributes) {
  await this.alertsService.evaluate(
    machine,
```

```

    attr.attributeName,
    attr.value,
    new Date(attr.timestamp),
  );
}

return { status: 'ok', inserted: attributeValues.length };

```

Listado 5.8. Fragmento de `IngestService.ingest`.

5.2. Descripción técnica de la implementación

Esta sección describe los aspectos técnicos más relevantes de la implementación realizada, las decisiones adoptadas durante el desarrollo y los principales retos encontrados durante la construcción del sistema.

5.2.1. Tecnologías y ejecución

El *backend* se implementa con TypeScript y NestJS. TypeORM traduce las entidades a tablas PostgreSQL y permite utilizar repositorios y consultas construidas desde el código. PostgreSQL se eligió por sus garantías relacionales y sus capacidades de consulta. El cliente se desarrolla con Angular e Ionic y se comunica con la API a través de HTTP; para las notificaciones se utiliza Socket.IO.

La aplicación NestJS arranca en el puerto 3000, activa CORS para el origen de desarrollo del cliente (<http://localhost:4200>) y registra el `ValidationPipe` global. La autenticación compara la contraseña recibida con el resumen *bcrypt* guardado y, si las credenciales son válidas, devuelve un JWT con una validez de un día.

5.2.2. Persistencia y despliegue de los servicios de datos

El fichero `tfg-ota/docker-compose.yml` define los servicios de infraestructura usados durante desarrollo y pruebas. Se despliega una instancia de PostgreSQL 15 con la base de datos `industrial_data` en el puerto 5432 y una instancia de pgAdmin en el puerto 5050. Los datos de PostgreSQL se almacenan en el volumen con nombre `pgdata`; por tanto, no se pierden al reiniciar o recrear el contenedor.

Esta configuración permite levantar el entorno con `docker compose up -d` sin instalar PostgreSQL ni pgAdmin de forma local. La API se ejecuta con Node.js durante el desarrollo y se conecta al puerto publicado por el contenedor. La separación es deliberada: Docker Compose encapsula los servicios de datos y el *backend* puede reiniciarse o depurarse independientemente. Docker aporta así un entorno reproducible para todo el equipo.

TypeORM tiene activada la opción `autoLoadEntities`, con lo que incorpora las entidades registradas por los módulos, y `synchronize`, que crea o adapta el esquema en el entorno de desarrollo. Esta última opción es útil durante la implementación, pero debe sustituirse por migraciones versionadas antes de un despliegue de producción para no aplicar cambios de esquema automáticamente.

5.2.3. Métricas recogidas y consultas

Una métrica es una observación numérica de un atributo de una máquina en un instante concreto. El sistema no limita los nombres de atributo: puede recibir, por ejemplo, temperatura, vibración, consumo o cualquier valor numérico que exponga la máquina. Cada registro conserva los siguientes datos:

- `attributeName`, `value` y `timestamp`, que identifican la medida.
- `machineId`, `connectorId` y `gatewayId`, que permiten localizar su origen en la jerarquía industrial.
- `lifeBit` y `connexionStatus`; en la ingesta actual se registran como `true` y `alive`, respectivamente, como evidencia de que la muestra fue recibida de un nodo operativo.

La entidad `AttributeValue` declara un índice compuesto sobre `machineId`, `attributeName` y `timestamp`. Este índice coincide con el patrón habitual de consulta: localizar el histórico de un atributo de una máquina y ordenarlo temporalmente. Además, en la petición del endpoint `GET /attribute-values/machine/:machineId/latest` se utiliza la operación `DISTINCT ON` de PostgreSQL para devolver el último valor de cada atributo de una máquina; es la consulta utilizada para una vista de estado actual.

El Listado 5.9 ilustra el contrato de ingesta y puede enviarse directamente al endpoint REST.

```
{
  "gatewayId": 1,
  "connectorId": 1,
  "machineId": 1,
  "attributes": [
    { "attributeName": "temperature", "value": 95.2,
      "timestamp": "2026-07-26T10:30:00.000Z" },
    { "attributeName": "vibration", "value": 8.4,
      "timestamp": "2026-07-26T10:30:00.000Z" }
  ]
}
```

Listado 5.9. Ejemplo de datos de ingesta.

Recurso	Método y ruta	Función
<i>Gateways</i>	POST/GET /gateways; GET/PATCH/DELETE /gateways/:id	Alta, listado paginado, consulta, modificación y borrado de nodos <i>edge</i> .
Conectores	POST/GET /connectors; GET/PATCH/DELETE /connectors/:id	Gestión de conectores asociados a un <i>gateway</i> .
Máquinas	POST/GET /machines; GET/PATCH/DELETE /machines/:id	Gestión de máquinas, con validación de su conector y <i>gateway</i> .
Valores	POST/GET /attribute-values; GET/PATCH/DELETE /attribute-values/:id	Consulta y gestión de valores de atributos almacenados.
Últimos valores	GET/attribute-values/machine/:machineId/latest	Devuelve la observación más reciente de cada atributo de una máquina.
Ingesta	POST /ingest/attributes	Recibe un lote jerárquico de métricas, lo persiste y evalúa sus reglas.
Alertas	GET/alerts/machine/:machineId ; GET /alerts/machine/:machineId /filter?severity=	Consulta las alertas de una máquina, opcionalmente filtradas por warning o critical .
Últimas alertas	GET /alerts/latest?limit=	Recupera las <i>N</i> alertas más recientes del sistema.
Usuarios y acceso	POST /users; POST /auth/login	Crea usuarios y devuelve un JWT tras comprobar las credenciales.

Cuadro 5.1: Endpoints REST implementados

5.2.4. API REST

El Cuadro 5.1 resume los endpoints implementados. Los recursos *gateway*, conector, máquina y valor de atributo siguen un patrón CRUD. En sus operaciones de listado se aceptan los parámetros `page`, `limit`, `orderBy`, `order` y `searchText`; el servicio aplica paginación, ordenación con campos permitidos y búsqueda textual antes de devolver los elementos y el total.

Las reglas de alerta se representan mediante la entidad **AlertRule**. En la versión implementada, la lógica de consulta y creación de alertas está expuesta mediante la API, mientras que la carga y modificación de reglas se realiza sobre la configuración persistida durante la preparación del entorno de pruebas. El diseño de la entidad deja aislada esta responsabilidad y permite incorporar posteriormente un controlador CRUD específico para las reglas sin modificar el flujo de ingesta.

5.2.5. Evaluación de umbrales y notificación

Cada regla contiene el atributo que vigila, el operador, el umbral, la severidad y la máquina a la que se aplica. Al recibir una métrica, el servicio recupera únicamente las reglas cuyo `machineId` y `attributeName` coinciden con la muestra. Después compara el valor frente al umbral con uno de los operadores `>`, `<`, `>=`, `<=` o `=`. Una regla “temperatura `>` 80”, por ejemplo, genera una alerta crítica cuando recibe el valor 95.2; si recibe 70 no crea ninguna alerta.

La alerta conserva tanto el valor recibido como la configuración que motivó su creación: atributo, operador, umbral, severidad, fecha y máquina. Conservar ambas partes es importante porque permite interpretar una alerta aunque la regla se modifique después. Tras guardar la alerta, el servidor emite el evento WebSocket **alert**; así, un cliente conectado puede actualizar la vista sin sondear continuamente la API.

Atributo	Valor	Regla	¿Alerta?	Resultado persistido
temperature	95.2	>80	Sí	Métrica y alerta crítica asociada a la máquina 1.
vibration	8.4	>= 8	Sí	Métrica y alerta configurada para la máquina 1.

Cuadro 5.2: Resultado del escenario de ingesta con dos umbrales superados

El Listado 5.10 muestra el código encargado de evaluar las métricas recibidas contra los umbrales.

```
if (this.matches(rule.operator, value, rule.threshold)) {
  const alert = this.alertRepo.create({
    attributeName, value, threshold: rule.threshold,
    operator: rule.operator, severity: rule.severity,
    timestamp, machine,
  });
  await this.alertRepo.save(alert);
  this.alertsGateway.emitAlert(alert);
}
```

Listado 5.10. Fragmento de `AlertService.evaluate`.

5.2.6. Resultados de la implementación

Siguiendo el ejemplo anterior, con dos atributos, la respuesta de ingesta es `{status: 'ok', inserted: 2}`. En el escenario donde existen las reglas `temperature >80` y `vibration >= 8`, los dos valores mostrados producen dos registros de alerta. El Cuadro 5.2 resume este caso.

5.3. Pruebas y validación

5.3.1. Estrategia de pruebas

La verificación se plantea en tres niveles complementarios. Las pruebas unitarias aíslan una clase de sus repositorios y de la red mediante dobles de prueba. Las pruebas de integración verifican la comunicación entre la API, TypeORM y PostgreSQL . Finalmente, las pruebas de sistema recorren el proceso completo, desde el envío de métricas hasta la consulta de alertas y la notificación en tiempo real.

El proyecto utiliza Jest para las pruebas unitarias y dispone también de configuración para pruebas *end-to-end*. La ejecución se realiza con `npm run test` para las unitarias y `npm run`

`test:e2e` para las pruebas de extremo a extremo.

5.3.2. Pruebas unitarias

Como ejemplo concreto se ha añadido una prueba sobre `AlertsService` (Ver Listado 5.11). La prueba sustituye los repositorios `TypeORM` y el *gateway* `WebSocket` por objetos simulados, prepara una regla `temperature >80` y evalúa una medición de 95. El objetivo es comprobar que el servicio persiste una alerta con los datos correctos y solicita su emisión, sin requerir `PostgreSQL` ni un servidor `Socket.IO`.

```
await service.evaluate(machine, 'temperature', 95, new Date('2026-01-01'));

expect(alertRepository.save).toHaveBeenCalledWith(
  expect.objectContaining({
    attributeName: 'temperature',
    value: 95,
    threshold: 80,
    severity: 'critical',
  }),
);
expect(alertsGateway.emitAlert).toHaveBeenCalledTimes(1);
```

Listado 5.11. Fragmento de testing de `AlertsService`.

El código completo de esta prueba se encuentra en `tfg-ota/src/alert/alert.service.spec.ts`. Este caso valida una regla de negocio crítica y complementa las pruebas de creación, consulta y actualización de los módulos de *gateways*, conectores, máquinas, métricas, usuarios y autenticación.

5.3.3. Pruebas de integración

Las pruebas de integración comprueban que los controladores trasladan los DTO a los servicios y que los servicios persisten las entidades y sus relaciones de forma correcta. El caso de mayor interés integra `IngestController`, `IngestService`, los repositorios de `Gateway`, `Connector`, `Machine` y `AttributeValue`, y `AlertsService`. Se verifica, además, que una jerarquía incoherente devuelve 404 Not Found en lugar de registrar una métrica asociada a un origen erróneo.

Al ejecutarse contra la base de datos levantada por Docker Compose, estas pruebas también validan el esquema generado por `TypeORM`, las claves foráneas y el índice de las métricas. Así se separan los errores de la lógica de umbrales de los posibles errores de mapeo o persistencia.

5.3.4. Pruebas de sistema y validación funcional

Para repetir el escenario completo se ha creado el script `tfg-ota/scripts/simulate-metrics.js`, ejecutable mediante `npm run simulate:metrics`. El script simula un nodo *edge*: envía periódicamente una muestra con los atributos `temperature` y `vibration` al endpoint `/ingest/attributes`. Los identificadores de *gateway*, conector y máquina, la URL y el intervalo se pueden ajustar mediante las variables `GATEWAY_ID`, `CONNECTOR_ID`, `MACHINE_ID`, `API_URL` e `INTERVAL_MS`.

El modo de funcionamiento se selecciona con `SIMULATOR_MODE`. El modo `normal` genera valores dentro de un intervalo habitual; `alert` genera valores elevados de manera continua; y `mixed`, configurado por defecto, alterna muestras normales con una muestra de alarma cada cuatro envíos. Gracias a esta configuración es posible comprobar tanto la ausencia de falsos positivos como la creación de alertas al cruzar los umbrales.

El procedimiento de validación funcional es el siguiente:

1. Levantar PostgreSQL y pgAdmin con Docker Compose e iniciar el *backend*.
2. Crear un *gateway*, un conector y una máquina; anotar sus identificadores.
3. Definir en la configuración de pruebas las reglas de los atributos que generará el simulador.
4. Ejecutar el simulador con los identificadores anteriores y con `SIMULATOR_MODE=mixed`.
5. Consultar `GET /attribute-values/machine/:machineId/latest` y `GET /alerts /machine/:machineId` para comprobar el histórico y las alertas generadas.
6. Abrir el cliente web conectado al WebSocket y verificar que recibe el evento `alert` para cada alerta creada.

5.3.5. Casos de prueba y resultados

El Cuadro 5.3 recoge los casos representativos de validación. Todos los resultados se derivan del comportamiento esperado del *backend* y se pueden repetir con el simulador descrito.

Id.	Caso	Tipo	Resultado esperado	Resultado
CP-01	Crear un <i>gateway</i> .	Integración	El elemento se guarda y aparece en el listado paginado.	Correcto.
CP-02	Crear conector y máquina con referencias existentes.	Integración	Se conservan las relaciones con el <i>gateway</i> y el conector.	Correcto.
CP-03	Enviar un lote válido con dos atributos.	Unitario	Respuesta <code>ok</code> , dos métricas insertadas y fechas preservadas.	Correcto.
CP-04	Enviar un valor que no supera el umbral.	Unitario	Se guarda la métrica y no se crea alerta.	Correcto.
CP-05	Enviar un valor que supera el umbral.	Unitario	Se guarda una alerta con valor, operador, umbral y severidad.	Correcto.
CP-06	Consultar últimos valores de una máquina.	Integración	Se devuelve una observación por cada atributo, la más reciente.	Correcto.
CP-07	Enviar una jerarquía no válida.	Unitario	La API rechaza la solicitud con <code>404</code> ; no se persisten valores.	Correcto.

Cuadro 5.3: Casos de prueba representativos

Los resultados confirman el recorrido completo de la información: la API acepta el lote estructurado, conserva el histórico, evalúa cada atributo con las reglas de su máquina, registra las incidencias y las publica a los clientes conectados. Este comportamiento satisface los requisitos de monitorización, consulta histórica y alerta definidos para el alcance implementado.

Capítulo 6

Conclusiones

La realización de este trabajo ha permitido consolidar y ampliar numerosos conocimientos adquiridos durante el Grado en Ingeniería Informática.

Desde el punto de vista técnico, se han aplicado conceptos relacionados con el desarrollo de aplicaciones web, el diseño de bases de datos relacionales, la arquitectura software, los sistemas distribuidos y las comunicaciones entre dispositivos.

Además, el proyecto ha supuesto una oportunidad para profundizar en tecnologías modernas como NestJS, TypeScript, PostgreSQL y Docker, así como en metodologías de desarrollo orientadas a la construcción de aplicaciones escalables y mantenibles.

6.1. Objetivos logrados

En relación con los objetivos definidos al inicio del proyecto, se ha desarrollado una plataforma funcional para centralizar la monitorización de la infraestructura industrial. La solución permite gestionar *gateways*, conectores y máquinas, recibir métricas mediante una API REST, almacenarlas junto con su contexto y marca temporal, consultar tanto el histórico como el último valor de cada atributo y generar alertas cuando se incumplen los umbrales configurados.

Estos resultados cubren los objetivos de recopilación, persistencia, consulta y análisis básico de la información, así como el objetivo de proporcionar una base web para la administración de la infraestructura. Además, el uso de una arquitectura modular, PostgreSQL y contenedores Docker responde a los requisitos de mantenibilidad, integridad y reproducibilidad del entorno; las pruebas y el simulador de métricas han permitido verificar el flujo completo de monitorización.

La funcionalidad de actualización remota *Over-The-Air* se planteó como objetivo inicial, pero no se incorporó en el alcance implementado por las limitaciones temporales, por lo que se mantiene como la principal línea de trabajo futuro.

6.2. Trabajo futuro

Aunque casi todos los objetivos principales del proyecto han sido alcanzados satisfactoriamente, existen diversas líneas de mejora que podrían incorporarse en futuras versiones del sistema.

Una de las funcionalidades previstas inicialmente que no pudo ser desarrollada debido a las limitaciones temporales del proyecto fue la implementación de un sistema de actualizaciones *Over-The-Air* (OTA). Este mecanismo permitiría desplegar nuevas versiones del software de los *gateway* de forma remota, simplificando las tareas de mantenimiento y reduciendo la necesidad de intervención manual sobre los dispositivos desplegados en planta.

Por otra parte, una línea de evolución especialmente interesante consistiría en incorporar técnicas de inteligencia artificial y aprendizaje automático para realizar tareas de mantenimiento predictivo. Aprovechando el histórico de métricas almacenado por la plataforma, sería posible entrenar modelos capaces de identificar patrones de comportamiento anómalos y estimar la probabilidad de fallo de una máquina antes de que éste se produzca.

Este enfoque permitiría anticipar necesidades de mantenimiento, reducir tiempos de inactividad y mejorar la eficiencia operativa de la infraestructura monitorizada.

6.3. Conclusiones personales

La realización de este Trabajo de Final de Grado ha supuesto una experiencia enriquecedora tanto desde el punto de vista académico como profesional, ya que me ha permitido aplicar de forma práctica conocimientos adquiridos a lo largo de la titulación y enfrentarme a problemas reales relacionados con el diseño y desarrollo de sistemas software complejos, además de ponerme en contacto con tecnologías actuales en un ámbito profesional real. También, me ha proporcionado una visión más cercana a los procesos habituales de desarrollo utilizados en entornos profesionales.

Además de los conocimientos técnicos adquiridos, este trabajo ha contribuido al desarrollo de capacidades relacionadas con la organización, la resolución de problemas, la toma de decisiones y la gestión del tiempo, competencias que resultarán de gran utilidad en futuros proyectos profesionales.

En conjunto, el proyecto ha permitido obtener una solución funcional y útil para el ámbito de la monitorización industrial, al mismo tiempo que a nivel personal ha supuesto una oportunidad para consolidar los conocimientos adquiridos durante el grado y prepararse para futuros retos profesionales.

Bibliografía

- [1] Atlassian. Roles de scrum. <https://www.atlassian.com/es/agile/scrum/roles>. Consultado el 4 de marzo de 2026.
- [2] Glassdoor. Sueldo de desarrollador de software sénior. https://www.glassdoor.es/Sueldos/desarrollador-de-software-senior-sueldo-SRCH_K00,32.htm. Consultado el 14 de abril de 2026.
- [3] IBM. What is a rest api? <https://www.ibm.com/think/topics/rest-apis>. Consultado el 10 de mayo de 2026.
- [4] IBM. ¿qué es postgresql? <https://www.ibm.com/es-es/think/topics/postgresql>. Consultado el 10 de mayo de 2026.
- [5] INCIBE-CERT. Opc ua: equilibrio entre ciberseguridad y rendimiento. <https://www.incibe.es/incibe-cert/blog/opc-ua-equilibrio-entre-ciberseguridad-y-rendimiento>. Consultado el 10 de mayo de 2026.
- [6] Ingeniería Plana Alta. Ingeniería informática. <https://iplapalletizers.com/ingenieria-informatica/>. Consultado el 17 de febrero de 2026.
- [7] Ingeniería Plana Alta. Quiénes somos. <https://iplapalletizers.com/quienes-somos/>. Consultado el 17 de febrero de 2026.
- [8] iTalenters. Sueldo de desarrollador full stack. <https://www.italenters.com/actualidad/sueldo-desarrollador-full-stack/>. Consultado el 14 de abril de 2026.
- [9] Kinsta. ¿qué es nestjs? <https://kinsta.com/es/blog/nestjs/>. Consultado el 12 de mayo de 2026.
- [10] Microsoft. Planes y precios de microsoft 365 para empresas. <https://www.microsoft.com/es-es/microsoft-365/business/microsoft-365-plans-and-pricing>. Consultado el 14 de abril de 2026.
- [11] MQTT.org. Mqtt: the standard for iot messaging. <https://mqtt.org/>. Consultado el 10 de mayo de 2026.
- [12] Omni Calculator. Calculadora de salario por hora. <https://www.omnicalculator.com/es/finanzas/salario-por-hora>. Consultado el 14 de abril de 2026.
- [13] Paessler. ¿qué es opc ua? <https://www.paessler.com/es/it-explained/opc-ua>. Consultado el 10 de mayo de 2026.

- [14] Profile. ¿qué es ionic? <https://profile.es/blog/que-es-ionic/>. Consultado el 12 de mayo de 2026.
- [15] Red Hat. ¿qué es docker? <https://www.redhat.com/es/topics/containers/what-is-docker>. Consultado el 8 de junio de 2026.
- [16] Universidad Internacional de La Rioja. Sueldo de desarrollador full stack. <https://www.unir.net/revista/ingenieria/sueldo-desarrollador-full-stack/>. Consultado el 14 de abril de 2026.
- [17] Universidad Internacional de La Rioja. ¿qué es angular? <https://www.unir.net/revista/ingenieria/que-es-angular/>. Consultado el 12 de mayo de 2026.